

Remerciements

Au terme de ce travail, je tiens à exprimer ma gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet de fin d'études.

Mes remerciements s'adressent d'abord à mon encadrant universitaire, le Professeur Ali Choukri, pour sa disponibilité, la pertinence de ses orientations et la confiance qu'il m'a accordée tout au long de ce travail. Ses exigences de rigueur ont durablement marqué ma façon d'aborder un problème d'ingénierie.

Je remercie également Monsieur Youssef Chadi, mon encadrant au sein de **Maxware Technology**, ainsi que Monsieur Moustapha Ouagague, directeur, et toute l'équipe, pour leur accueil, leur accompagnement technique au quotidien et la liberté qu'ils m'ont laissée dans mes choix d'architecture. Travailler sur un produit destiné à de vrais utilisateurs a donné tout son sens à ce projet.

Ma reconnaissance va aussi aux membres du jury, qui ont accepté d'évaluer ce mémoire, ainsi qu'au corps professoral de la filière Ingénierie des Systèmes Intelligents de la Faculté des Sciences de Kénitra, pour la qualité de la formation reçue.

Enfin, je remercie ma famille pour son soutien constant et sa patience durant ces mois de travail intensif. Ce mémoire leur doit beaucoup.

Résumé

Les progiciels de gestion intégrés (ERP) tels qu'Odoo concentrent l'essentiel des processus métier des entreprises, mais leur richesse fonctionnelle s'accompagne d'une courbe d'apprentissage importante : navigation dense, documentation dispersée et absence d'assistance contextuelle pénalisent l'adoption et la productivité des utilisateurs.

Ce mémoire présente la conception et la réalisation, au sein de la société **Maxware Technology**, d'un **Système Multi-Agent Intelligent**, intégré nativement à Odoo 19. Huit agents spécialisés (orchestration, contexte, guidage, documentation, formation, traduction, analyse et prédiction), coordonnés par un graphe d'états LangGraph et exposés via une API FastAPI, collaborent pour offrir une assistance proactive : réponses ancrées dans la documentation officielle grâce à un pipeline RAG (ChromaDB, sentence-transformers), guidage visuel pas-à-pas sur l'interface (surbrillance, info-bulles, flèches), mode démonstration, formation adaptative fondée sur la répétition espacée SM-2, support multilingue et suggestions proactives.

Le système repose sur une architecture découplée et sécurisée : un module Odoo (frontend OWL) communique avec le backend conteneurisé au travers d'un proxy serveur qui préserve les sessions et les ACLs, tandis qu'une couche LLM multi-fournisseur (NVIDIA NIM, OpenAI, Azure, Ollama) protégée par un circuit breaker garantit la portabilité et la résilience. Le streaming SSE de bout en bout réduit fortement la latence perçue en diffusant en direct les étapes du pipeline et les jetons de la réponse.

La validation s'appuie sur 294 tests automatisés en intégration continue (couverture de code de 67 %), des tests de bout en bout Playwright couvrant les cinq flux fonctionnels majeurs, une évaluation quantitative du pipeline documentaire (Precision@5 globale de 0,843 et MRR de 0,973 sur 65 questions couvrant douze modules) et des mesures de performance détaillées : les optimisations appliquées (court-circuits heuristiques de classification, caches Redis, inférence de guidage déterministe) ramènent par exemple le guidage explicite de 9–14 s à 3,4–5,5 s et les requêtes documentaires répétées à un temps quasi nul.

Mots-clés : Système multi-agent, Odoo 19, LLM, LangGraph, RAG, FastAPI, OWL, assistance contextuelle, formation adaptative, streaming SSE.

Liste des abréviations

Sigle	Signification
ACL	Access Control List (liste de contrôle d'accès Odoo)
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DOM	Document Object Model
ERP	Enterprise Resource Planning (progiciel de gestion intégré)
HPA	Horizontal Pod Autoscaler (Kubernetes)
HTTP(S)	HyperText Transfer Protocol (Secure)
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model (grand modèle de langage)
MAS	Multi-Agent System (système multi-agent)
NIM	NVIDIA Inference Microservices
NLP	Natural Language Processing (traitement du langage naturel)
ORM	Object-Relational Mapping
OWL	Odoo Web Library
PFE	Projet de Fin d'Études
QCM	Questionnaire à Choix Multiples
RAG	Retrieval-Augmented Generation (génération augmentée par récupération)
REST	Representational State Transfer
RPC	Remote Procedure Call
SM-2	SuperMemo 2 (algorithme de répétition espacée)
SSE	Server-Sent Events (événements envoyés par le serveur)
STT / TTS	Speech-To-Text / Text-To-Speech
TLS	Transport Layer Security
TTL	Time To Live (durée de vie d'une entrée de cache)
UI	User Interface (interface utilisateur)

Table des matières

Remerciements	3
Résumé	4
Liste des abréviations	5
Introduction générale	8
Présentation de l'organisme d'accueil	9
Chapitre I : Analyse du projet	10
I.1 Étude de l'existant	10
I.1.1 Comparaison des solutions d'assistance IA	10
I.1.2 Problèmes posés et solutions proposées	12
I.1.3 Planification du projet	13
I.2 Analyse des besoins	14
I.2.1 Besoins fonctionnels	14
I.2.2 Besoins non fonctionnels	15
I.2.3 Contraintes techniques	16
I.3 Spécifications détaillées	17
I.3.1 Spécifications fonctionnelles	17
I.3.2 Spécifications techniques	17
I.3.3 Architecture multi-agent proposée	18
Conclusion	19
Chapitre II : Conception	20
II.1 Introduction	20
II.2 Diagramme C4 — Niveau 1 : contexte système	20
II.2.1 Présentation du modèle C4	20
II.2.2 Diagramme de contexte (niveau 1)	20
II.3 Diagramme C4 — Niveau 2 : conteneurs	22
II.3.1 Vue d'ensemble des conteneurs	22
II.3.2 Description détaillée des conteneurs	22

II.4 Diagramme C4 — Niveau 3 : composants	24
II.4.1 Spécification des huit agents spécialisés	24
II.4.2 Architecture interne de chaque agent	26
II.4.3 Composants transversaux	27
II.5 Flux de données	28
II.5.1 Flux principal — question utilisateur	28
II.5.2 Flux de formation adaptative	29
II.5.3 Flux de mode démonstration	30
II.5.4 Flux de détection et de résolution d'erreur	31
II.5.5 Flux RAG — pipeline de récupération documentaire	32
II.6 Communication inter-agents	33
II.6.1 Protocoles de communication	33
II.6.2 Matrice des échanges	33
II.6.3 Protocole de message standard	34
II.7 Choix technologiques et stack complète	34
II.7.1 Vue d'ensemble de la stack	34
II.7.2 Justification des choix clés	35
II.7.3 Stratégie de déploiement	36
Conclusion	37
Chapitre III : Réalisation	38
III.1 Environnement de développement et outils	38
III.2 Implémentation du backend multi-agent	39
III.2.1 Orchestrateur (LangGraph)	39
III.2.2 Les agents spécialisés	39
III.2.3 Couche LLM multi-fournisseur et résilience	40
III.2.4 Pipeline RAG	40
III.2.5 Communication inter-agents	41
III.2.6 Streaming SSE de bout en bout	41
III.2.7 Contrat d'API du backend	41
III.3 Implémentation du module Odoo (frontend OWL)	42

III.3.1 Avatar et communication	42
III.3.2 Moteur de guidage visuel	46
III.3.3 Mode démonstration	52
III.3.4 Formation adaptative	53
III.3.5 Tableaux de bord d'administration et de suivi	56
III.3.6 Proxy et sécurité de bout en bout	62
III.4 Déploiement et intégration continue	63
Conclusion	64
Chapitre IV : Tests et évaluation	65
IV.1 Stratégie de tests	65
IV.2 Résultats des tests fonctionnels	65
IV.3 Mesures de performance	68
IV.3.1 Latence de bout en bout par flux	68
IV.3.2 Optimisations appliquées et gains mesurés	68
IV.4 Évaluation quantitative du pipeline documentaire (RAG)	69
IV.4.1 Protocole d'évaluation	69
IV.4.2 Résultats	70
IV.4.3 Analyse des résultats	71
IV.4.4 Axes d'amélioration	72
IV.5 Limites observées et pistes d'amélioration	72
IV.6 Bilan de conformité au cahier des charges	73
Conclusion	74
Conclusion générale et perspectives	75
Bibliographie	77

Table des figures

1	Diagramme C4 de niveau 1 — contexte du Système Multi-Agent Intelligent	21
2	Diagramme C4 de niveau 2 — conteneurs du système	22
3	Diagramme C4 de niveau 3 — composants de la couche agents	25
4	Avatar flottant Maxware superposé à l'accueil d'Odoo 19, panneau fermé	43
5	Interface de chat de l'assistant Maxware (widget OWL flottant)	43
6	Réponse rendue en Markdown, avec badges de traçabilité (agent et fournisseur LLM)	44
7	Saisie vocale active : micro en écoute et retour visuel « Je vous écoute... »	44
8	Question posée en anglais : détection de la langue et réponse traduite par le LangAgent	45
9	Bandeau de suggestions proactives alimenté par le PredictAgent	45
10	Panneau « Documentation dynamique » : sources classées avec leur score de pertinence	46
11	Tutoriels vidéo YouTube recommandés aux côtés de la documentation Odoo	46
12	Guidage ancré : sur un nouveau devis, l'étape cible le champ réel « Client » (surbrillance), avec le pas « Étape 3/9 — Renseigner « Client » » et l'étape suivante annoncée.	48
13	Guidage pas-à-pas avec progression diffusée en direct (streaming SSE)	49
14	Mode surbrillance : le bouton cible est entouré d'un anneau animé	49
15	Mode info-bulle : bulle d'aide contextuelle ancrée sur le champ « Client »	50
16	Mode flèche : flèche SVG animée pointant la cible (surbrillance ajoutée pour la lisibilité du cliché)	50
17	Guidage pas-à-pas en cours : compteur « Étape 2/5 » dans la barre d'état du panneau	51

18	Résolution d'erreur : champs requis signalés, diagnostic en trois points et documentation proposée par l'avatar	52
19	Mode démonstration : bandeau d'étape, cible mise en évidence, plan des étapes et confirmation explicite	53
20	Évaluation par QCM dans une boîte de dialogue OWL dédiée : question, choix multiples et progression	54
21	Synthèse de fin de QCM : score noté côté serveur et questions à revoir	54
22	Étape d'un parcours de formation adaptatif, avec déclenchement du guidage (« Montre-moi ») et de la ressource vidéo	55
23	Menu racine « Maxware AI » : configuration, personnalisation, Insights, apprentissage et supervision des agents	57
24	Page Configuration : connexion au backend (clé masquée) et paramètres du fournisseur LLM	58
25	Personnalisation de l'avatar avec aperçu en direct (apparence, position, personnalité)	58
26	Tableau de bord « Insights » : journal d'usage issu de l'AnalyticsAgent	59
27	Tableau de bord de supervision des agents dans Odoo	59
28	Tableau de bord « Mon niveau » du module Apprentissage (états SM-2 par module)	60
29	Historique des QCM : score, taux de réussite et niveau dérivé de l'état SM-2	61
30	Apprentissage > Succès : badges et progression de l'utilisateur	61
31	Apprentissage > Statistiques : activité et maîtrise par module	62
32	Exécution locale de la suite pytest avec couverture (Windows : 289 réussis, 5 ignorés faute de Redis joignable, 66 %)	67
33	Récapitulatif des cinq flux fonctionnels validés de bout en bout (session Playwright)	67

Introduction générale

À l'ère de la transformation digitale, les progiciels de gestion intégrés (ERP) comme Odoo sont devenus la pierre angulaire de la performance des entreprises. Il existe cependant un fossé croissant entre la complexité de ces systèmes d'information et la capacité des utilisateurs à les exploiter pleinement : les ERP traditionnels reposent encore sur des workflows statiques qui engendrent une courbe d'apprentissage abrupte et une surcharge cognitive importante.

L'émergence de l'intelligence artificielle générative, portée par l'architecture Transformer [9], et plus particulièrement des systèmes multi-agents (MAS) [1], ouvre une nouvelle ère : celle de l'ERP agentique. Contrairement aux assistants classiques (chatbots) qui se contentent de répondre à des requêtes textuelles, un système agentique est capable d'autonomie, de perception contextuelle et d'action directe sur l'interface logicielle.

Ce projet de fin d'études, réalisé au sein de la société **Maxware Technology**, a pour objectif de concevoir et de développer un Système Multi-Agent Intelligent intégré nativement à la plateforme Odoo 19. En nous inscrivant dans la lignée des travaux pionniers appliquant les systèmes multi-agents aux ERP [2, 3] et de leur prolongement récent par les architectures multi-agents fondées sur les grands modèles de langage [4], nous proposons une architecture où une constellation de huit agents spécialisés collabore pour transformer l'expérience utilisateur : le système ne se contente pas d'informer, il guide visuellement, démontre, diagnostique les erreurs en temps réel et propose un apprentissage adaptatif personnalisé.

Ce mémoire s'articule autour de quatre chapitres :

- **Chapitre I — Analyse du projet** : étude de l'état de l'art de l'assistance intelligente dans les ERP, analyse des besoins, contraintes et spécifications fonctionnelles et techniques ;
- **Chapitre II — Conception** : modélisation de l'architecture multi-agent selon le modèle C4, description des flux de données, de la communication inter-agents et justification des choix technologiques ;
- **Chapitre III — Réalisation** : implémentation du backend multi-agent (FastAPI, LangGraph, RAG, streaming SSE) et du module Odoo (OWL), sécurité, déploiement et intégration continue ;
- **Chapitre IV — Tests et évaluation** : stratégie de validation, résultats fonctionnels, mesures de performance, optimisations appliquées et limites observées.

Par cette approche, nous ambitionnons de transformer l'ERP d'un outil complexe en un environnement de travail proactif, augmentant ainsi la productivité et l'autonomie des collaborateurs.

Présentation de l'organisme d'accueil



Ce projet de fin d'études a été réalisé au sein de Maxware Technology, société de services en ingénierie informatique basée à Kénitra, spécialisée dans le développement de solutions informatiques sur mesure. L'entreprise privilégie un engagement forfaitaire qui garantit à ses clients une maîtrise des risques et des coûts, dans une logique d'accompagnement de bout en bout, du besoin jusqu'au déploiement. Son offre s'articule autour de quatre pôles : le développement ERP sur la plateforme Odoo (de l'auto-entrepreneur à la TPE/PME et aux associations), le conseil (changements organisationnels et structurels, gestion financière et des risques), la création de sites web (du site vitrine au développement sur mesure) et le développement d'applications mobiles (natives ou hybrides). C'est dans ce cadre, un acteur de l'ingénierie logicielle et de l'intégration Odoo cherchant à différencier ses déploiements par une assistance intelligente, que s'inscrit ce stage : la conception et la réalisation d'un système multi-agent d'assistance contextuelle pour Odoo 19, visant à réduire la courbe d'apprentissage des utilisateurs des ERP livrés par l'entreprise.

Chapitre I : Analyse du projet

I.1 Étude de l'existant

Les systèmes ERP (Enterprise Resource Planning) constituent aujourd'hui le cœur des systèmes d'information des entreprises modernes. Parmi les solutions leaders du marché figurent SAP, Oracle ERP Cloud, Microsoft Dynamics 365 et Odoo. Ces plateformes centralisent l'ensemble des processus métier : comptabilité, ventes, achats, stocks, production, ressources humaines, et bien d'autres.

Odoo : une solution open source modulaire

Odoo se distingue par son approche modulaire et open source. Avec plus de 30 000 modules disponibles et une communauté active de plusieurs millions d'utilisateurs, Odoo est particulièrement apprécié pour sa flexibilité, l'absence de coût de licence de son édition Community et son large écosystème communautaire. La version 19, récemment publiée, apporte des améliorations significatives en matière de performance et d'ergonomie grâce au framework OWL (Odoo Web Library) [19, 20].

Limitations actuelles des ERP

Malgré leurs capacités fonctionnelles étendues, les ERP actuels présentent des défis majeurs en matière d'utilisabilité :

- **Complexité d'apprentissage** : la prise en main demande souvent plusieurs mois de formation pour maîtriser les fonctionnalités de base, ce qui génère des coûts élevés et ralentit l'adoption ;
- **Navigation difficile** : des interfaces riches mais denses désorientent les nouveaux utilisateurs ; la multiplicité des menus, vues et actions crée une surcharge cognitive importante ;
- **Documentation dispersée** : la documentation est fragmentée entre wikis, forums, vidéos et documentation officielle, rendant difficile la recherche d'information contextuelle ;
- **Absence d'assistance contextuelle** : les utilisateurs sont livrés à eux-mêmes face aux erreurs ou aux tâches complexes, sans guidage adaptatif en temps réel ;
- **Support réactif uniquement** : les équipes de support interviennent après l'apparition d'un problème, plutôt que de prévenir les difficultés par une approche proactive.

I.1.1 Comparaison des solutions d'assistance IA

Plusieurs solutions émergentes tentent d'améliorer l'expérience utilisateur des applications d'entreprise par l'intelligence artificielle. Le tableau ci-dessous compare les

principales approches :

Solution	Approche	Limites
Chatbots traditionnels	Réponses fondées sur des FAQ et des règles prédéfinies, intégration dans l'interface	Rigides, ne comprennent pas le langage naturel complexe, limités aux scénarios prévus
Assistants LLM génériques	ChatGPT, Claude intégrés via API, réponses générales	Pas de contexte Odoo spécifique, hallucinations possibles, aucun guidage visuel
Copilot Microsoft 365	IA intégrée dans Office, aide à la rédaction, suggestions contextuelles	Limité à l'écosystème Microsoft, pas de formation adaptative
Odoo 19 AI Agent (natif)	Assistant textuel intégré utilisant des « Tools » et des « Sources »	Interface uniquement textuelle, passif (attend la question), pas de guidage visuel sur l'UI

Le tableau suivant resserre la comparaison sur quatre critères discriminants, en y ajoutant les assistants natifs des grands éditeurs ¹ :

Solution	Guidage visuel	Formation adaptative	ERP cible (intégration)	Multi-agent
Chatbots traditionnels	Non	Non	Variable (ad hoc)	Non
Assistants LLM génériques	Non	Non	Aucun ERP	Non
SAP Joule	Non (conversationnel)	Non	SAP S/4HANA (native)	Émergent
Oracle Digital Assistant	Non	Non	Oracle Fusion Cloud (native)	Non

1. Capacités relevées dans la documentation publique des éditeurs (SAP Joule, Oracle Digital Assistant, Salesforce Einstein Copilot/Agentforce, Microsoft Copilot pour Dynamics 365), consultée en juin 2026. « Émergent » signale des capacités agentiques annoncées ou en cours de déploiement.

Solution	Guidage visuel	Formation adapta- tive	ERP cible (intégration)	Multi- agent
Salesforce Einstein Copilot / Agentforce	Non	Non	Salesforce CRM (native)	Émergent
Microsoft Copilot pour Dynamics 365	Non	Non	Dynamics 365 (native)	Émergent
Odoo 19 AI Agent (natif)	Non	Non	Odoo 19 (native)	Non
Maxware (notre système)	Oui (3 modes)	Oui (QCM + SM-2)	Odoo 19 (native)	Oui (8 agents)

La lecture croisée de ces deux tableaux précise notre positionnement. Les grands éditeurs intègrent désormais des copilotes conversationnels à leurs ERP, et plusieurs annoncent des capacités agentiques (Joule, Agentforce, agents Dynamics 365) : sur ce plan, l'assistance par IA n'est plus une singularité. Aucune de ces solutions ne propose en revanche, à notre connaissance, la combinaison qui fait l'objet de ce mémoire — un guidage visuel superposé à l'interface réelle, une formation adaptative fondée sur la répétition espacée et une architecture multi-agent spécialisée, le tout intégré nativement à un ERP open source. Ce constat rejoint les perspectives ouvertes par les travaux académiques sur l'application des systèmes multi-agents aux ERP [2, 3] et, plus récemment, par les études consacrées aux systèmes multi-agents fondés sur les LLM [4] et à leur intégration dans les processus d'entreprise [5]², et délimite précisément la contribution visée.

I.1.2 Problèmes posés et solutions proposées

2. Côté praticiens, on peut également mentionner le retour d'expérience de C. Simon, « Beyond Traditional ERP : Implementing Multi-Agent Systems in Odoo 18 », article LinkedIn, 2025 — source non académique, citée à titre d'illustration de l'intérêt industriel pour cette approche.

Problème identifié	Solution proposée
Courbe d'apprentissage importante nécessitant plusieurs mois de formation	Agent de formation personnalisée (LearnAgent) avec parcours adaptatifs, QCM intelligents et répétition espacée SM-2
Utilisateurs perdus dans l'interface complexe et les multiples menus	Agent de guidage (GuideAgent) + guidage visuel : surbrillance CSS, info-bulles, flèches animées
Difficulté à comprendre comment réaliser une tâche spécifique	Mode démonstration : l'avatar exécute les actions étape par étape avec narration
Documentation dispersée et non contextualisée	Agent documentaire (DocAgent) avec RAG sur la documentation Odoo et vidéos YouTube indexées
Messages d'erreur cryptiques et difficiles à résoudre	Agent de contexte (ContextAgent) analysant l'erreur et déclenchant une résolution guidée fondée sur l'historique
Barrière linguistique pour les utilisateurs internationaux	Agent de traduction (LangAgent) : traduction en temps réel, support vocal multilingue
Fonctionnalités sous-utilisées	Agent d'analyse (AnalyticsAgent) détectant les schémas d'usage et les points de friction
Tâches répétitives et besoins non anticipés	Agent de prédiction (PredictAgent) anticipant les besoins et formulant des suggestions proactives

I.1.3 Planification du projet

Conformément au planning prévisionnel du cahier des charges, le projet s'étale sur six mois (24 semaines) et se décompose en six phases majeures :

Phase	Durée	Activités principales	Livrables
Phase 1	Sem. 1–2	Étude de l'existant, analyse des besoins, spécifications	Rapport d'analyse, cahier des charges

Phase	Durée	Activités principales	Livrables
Phase 2	Sem. 3–6	Conception de l'architecture, choix technologiques	Dossier de conception, diagrammes C4, flux de données complets
Phase 3	Sem. 7–14	Développement des agents de base, pipeline RAG	Trois agents opérationnels, corpus indexé
Phase 4	Sem. 15–18	Développement des agents additionnels, avatar, intégration Odoo	Huit agents complets, formation adaptative, multilinguisme
Phase 5	Sem. 19–22	Tests, optimisation des performances, streaming SSE, durcissement sécurité	Système testé et validé, documentation technique
Phase 6	Sem. 23–24	Rédaction du mémoire, préparation de la soutenance	Mémoire complet, présentation, démonstration

I.2 Analyse des besoins

L'analyse qui suit s'appuie sur le cahier des charges établi par Maxware Technology au lancement du stage. Celui-ci fixe cinq objectifs : concevoir une architecture multi-agent collaborative s'intégrant nativement à Odoo ; développer un avatar interactif capable de communiquer avec l'utilisateur ; permettre une assistance contextuelle en surimpression sur l'interface ; créer un système de gestion et de recommandation de documentation intelligente ; et proposer une expérience utilisateur fluide et non intrusive. Les besoins détaillés ci-dessous précisent et étendent ces objectifs ; leur couverture finale est dressée au chapitre IV (section IV.6).

I.2.1 Besoins fonctionnels

Avatar interactif et communication naturelle

- Affichage d'un avatar animé et personnalisable, flottant au-dessus de toutes les pages Odoo ;
- Communication textuelle via une interface de chat ;
- Communication vocale bidirectionnelle (reconnaissance et synthèse) ;
- Compréhension du langage naturel avec détection d'intention.

Guidage visuel contextuel

- Surbrillance automatique des éléments d'interface pertinents ;
- Affichage d'info-bulles contextuelles ;
- Flèches animées pointant vers les éléments cibles.

Mode démonstration automatique

- Exécution pas-à-pas des actions par l'avatar sur l'interface réelle ;
- Narration expliquant chaque étape ;
- Confirmation utilisateur entre les étapes pour une exécution sûre.

Documentation dynamique et intelligente

- Recherche sémantique dans la documentation officielle Odoo ;
- Recommandation proactive d'articles pertinents, ancrée dans les sources ;
- Intégration et indexation de tutoriels vidéo YouTube.

Formation personnalisée et adaptative

- Évaluation initiale du niveau par QCM générés dynamiquement ;
- Génération de parcours de formation adaptatifs ;
- Ajustement de la difficulté par répétition espacée (algorithme SM-2) ;
- Suivi de progression avec visualisation dans des tableaux de bord Odoo.

Support multilingue

- Détection de la langue et traduction en temps réel des conversations ;
- Commandes vocales dans plusieurs langues.

Analytics et proactivité

- Détection des points de friction et des erreurs récurrentes ;
- Identification des fonctionnalités sous-utilisées ;
- Prédiction des besoins de l'utilisateur et suggestions proactives ;
- Supervision de l'activité des agents (tableaux de bord d'administration).

I.2.2 Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse maîtrisé pour les interactions courantes ; réduction de la latence perçue par diffusion progressive de la réponse (streaming SSE)

Catégorie	Exigence
Scalabilité	Backend sans état (état de session externalisé dans Redis) autorisant la montée en charge horizontale derrière un répartiteur de charge
Disponibilité	Haute disponibilité pendant les heures opérationnelles ; dégradation gracieuse en cas de panne d'un composant (LLM, Redis)
Sécurité	Authentification par session Odoo native et clé API (jeton Bearer) ; respect strict des ACLs Odoo ; chiffrement HTTPS ; aucune clé exposée au navigateur ; journal d'audit
Maintenabilité	Code documenté, architecture modulaire, contrats de données typés, suite de tests automatisée et intégration continue
Compatibilité	Support d'Odoo 19 Community et Enterprise ; indépendance vis-à-vis du fournisseur de LLM

I.2.3 Contraintes techniques

Contraintes d'intégration Odoo

- Développement sous forme de module Odoo respectant la structure standard (manifeste, vues, sécurité) ;
- Utilisation du framework OWL pour les composants frontend ;
- Respect du système de permissions et des ACLs existants (accès aux données exclusivement via l'ORM côté serveur).

Contraintes technologiques

- Backend : Python, FastAPI, Redis ;
- Multi-agents : LangChain / LangGraph ;
- LLM : OpenAI, Claude ou Ollama (local) activables par configuration ;
- Base vectorielle : ChromaDB pour le pipeline RAG ;
- Avatar : animations Lottie (2D) embarquées localement.

Contraintes de déploiement

- Architecture de microservices découplée via Docker et Docker Compose ;
- Intégration et déploiement continu avec GitHub Actions ;
- Trajectoire de migration vers Kubernetes pour la production.

Contraintes de coûts

- Optimisation des appels API LLM : court-circuits heuristiques, mise en cache agressive des résultats RAG et limitation des tentatives ;
- Modèles d'embedding légers exécutés localement (pas de coût d'API pour la vectorisation).

I.3 Spécifications détaillées

I.3.1 Spécifications fonctionnelles

Avatar interactif

- Avatar affiché en position flottante et déplaçable (drag-and-drop) ;
- Animations émotionnelles : repos, réflexion, parole, célébration.

Guidage visuel contextuel

- Identification sémantique des éléments d'interface cibles ;
- Surbrillance avec bordures animées ;
- Info-bulles en position adaptative ;
- Avancement piloté par l'action réelle de l'utilisateur (clic sur la cible ou changement d'écran).

Mode démonstration

- Décomposition d'une tâche en actions atomiques exécutées séquentiellement ;
- Narration avant chaque action ;
- Confirmation explicite de l'utilisateur entre les étapes.

Formation adaptative

- Génération de QCM pour l'évaluation initiale et la validation des acquis ;
- Parcours adaptatif ancré dans la documentation réelle (RAG) ;
- Notation côté serveur (les réponses correctes ne transitent jamais vers le client) ;
- Visualisation de la progression et de l'historique dans Odoo.

I.3.2 Spécifications techniques

Composant	Technologie	Justification
Backend agents	Python 3.11, FastAPI	Performance async/await, écosystème IA
Orchestration multi-agents	LangChain / LangGraph	Frameworks matures, orchestration par graphe d'états

Composant	Technologie	Justification
LLM	OpenAI / Claude / Ollama configurables	Indépendance fournisseur ; cloud en production, local en développement
Base vectorielle	ChromaDB	RAG performant, recherche sémantique, installation locale
Cache & bus de messages	Redis 7	Sessions, cache, pub/sub, streams
Frontend avatar	Framework OWL	Intégration native Odoo 19
Traduction	LLM multilingue (DeepL optionnel)	Qualité de traduction, coût maîtrisé
Vocal STT/TTS	Web Speech API du navigateur	Aucune dépendance cloud obligatoire
Documentation	Corpus Odoo (RST/Markdown), YouTube Data API	Sources officielles et tutoriels vidéo
Conteneurisation	Docker, Docker Compose, manifestes Kubernetes	Isolation, reproductibilité, montée en charge
Observabilité	Prometheus, Grafana	Métriques d'exploitation en temps réel

I.3.3 Architecture multi-agent proposée

L'architecture repose sur huit agents spécialisés, dont un agent orchestrateur qui coordonne les sept autres [1, 2] :

Agent	Rôle principal	Capacités clés
OrchestratorAgent	Classifier l'intention et router les requêtes vers les agents appropriés	Graphe d'états, gestion des priorités, workflows parallèles
ContextAgent	Analyser la position de l'utilisateur dans l'interface	Événements DOM, historique de navigation, profil utilisateur
GuideAgent	Répondre aux questions et guider pas-à-pas	NLP, génération de réponses, séquences de guidage visuel, mode démonstration

Agent	Rôle principal	Capacités clés
DocAgent	Gérer et recommander la documentation	Pipeline RAG, indexation de vidéos YouTube
LearnAgent	Adapter l'assistance au profil de l'utilisateur	Parcours adaptatifs, QCM, répétition espacée SM-2
LangAgent	Assistance multilingue	Détection de langue, traduction temps réel, indications vocales
AnalyticsAgent	Collecter les données d'usage	Télémétrie, détection des frictions et de la sous-utilisation
PredictAgent	Anticiper les besoins	Heuristiques comportementales, suggestions proactives

Conclusion

L'analyse du projet révèle un besoin critique d'amélioration de l'expérience utilisateur dans les systèmes ERP, particulièrement dans Odoo : courbe d'apprentissage importante, navigation complexe et absence d'assistance contextuelle intelligente.

L'étude comparative des solutions existantes démontre qu'aucune n'offre une approche complète combinant avatar interactif, guidage visuel, formation personnalisée, mode démonstration et support multilingue. Le système multi-agent proposé répond directement à ces limitations avec huit agents spécialisés orchestrés intelligemment.

L'architecture technique s'appuie sur des technologies éprouvées (Python, FastAPI, LangChain/LangGraph, ChromaDB, Redis, OWL) garantissant la faisabilité du projet, et le planning en six phases sur 24 semaines permet un développement itératif avec validation continue. Les spécifications établies dans ce chapitre constituent la base de la phase de conception détaillée présentée au chapitre suivant.

Chapitre II : Conception

II.1 Introduction

Après l'analyse des besoins réalisée dans le chapitre précédent, cette partie est consacrée à la conception du système proposé. L'objectif est de définir une architecture technique capable de répondre aux exigences identifiées.

Pour cela, nous adoptons une approche fondée sur une architecture multi-agent, où plusieurs agents spécialisés collaborent pour assurer une assistance intelligente dans Odoo. La conception s'appuie sur le modèle C4 [21] afin de représenter le système à différents niveaux d'abstraction, ainsi que sur la description des flux de données pour illustrer le fonctionnement global. Ce chapitre présente l'architecture du système, les interactions entre ses composants, ainsi que les principaux choix technologiques retenus.

II.2 Diagramme C4 — Niveau 1 : contexte système

II.2.1 Présentation du modèle C4

Le modèle C4, développé par Simon Brown [21], offre une approche hiérarchique pour documenter l'architecture logicielle à travers quatre niveaux d'abstraction : Context (contexte), Containers (conteneurs), Components (composants) et Code. Cette approche permet à différentes parties prenantes (décideurs, développeurs, architectes) de comprendre l'architecture au niveau de détail qui leur est pertinent. Nous exploitons ici les trois premiers niveaux (C1, C2, C3), qui suffisent à donner une vision complète du système.

II.2.2 Diagramme de contexte (niveau 1)

Le diagramme de contexte positionne le Système Multi-Agent Intelligent dans son écosystème, en identifiant les acteurs externes (utilisateurs, systèmes tiers) qui interagissent avec lui.

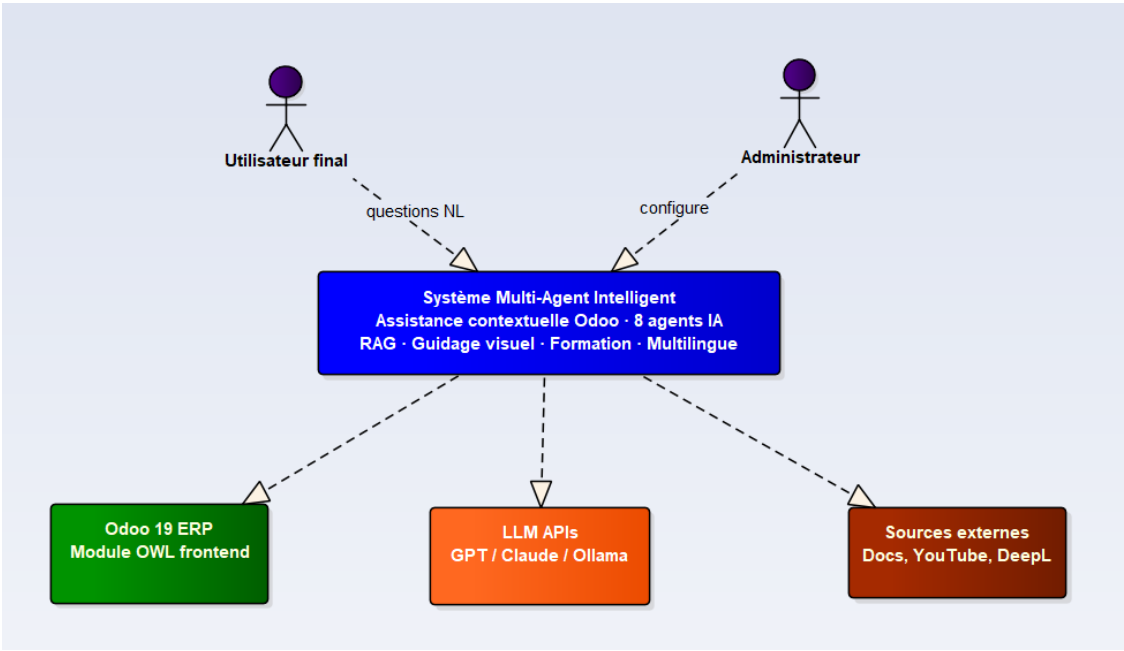


FIGURE 1 – Diagramme C4 de niveau 1 — contexte du Système Multi-Agent Intelligent

Acteurs identifiés

Acteur	Type	Interactions avec le système
Utilisateur final	Acteur humain principal	Saisit des questions en langage naturel, reçoit le guidage visuel, suit les formations adaptatives, utilise les commandes vocales
Administrateur	Acteur humain secondaire	Configure les agents et la connexion au backend, personnalise l’avatar, supervise les tableaux de bord (Agents, Insights, Apprentissage)
Odoo 19 ERP	Système externe principal	Fournit le contexte métier (utilisateur, module actif, données via l’ORM) ; héberge le module frontend et le proxy serveur
LLM APIs	Système externe IA	NVIDIA NIM par défaut pour la génération ; OpenAI, Claude ou Ollama activables par configuration
Sources documentaires	Système externe data	Documentation officielle Odoo, YouTube Data API (tutoriels vidéo), DeepL (traduction, optionnel)

II.3 Diagramme C4 — Niveau 2 : conteneurs

II.3.1 Vue d'ensemble des conteneurs

Le diagramme de conteneurs décompose le Système Multi-Agent Intelligent en ses grandes unités déployables et exécutables. Chaque conteneur est un processus ou une application distincte communiquant via des protocoles réseau définis.

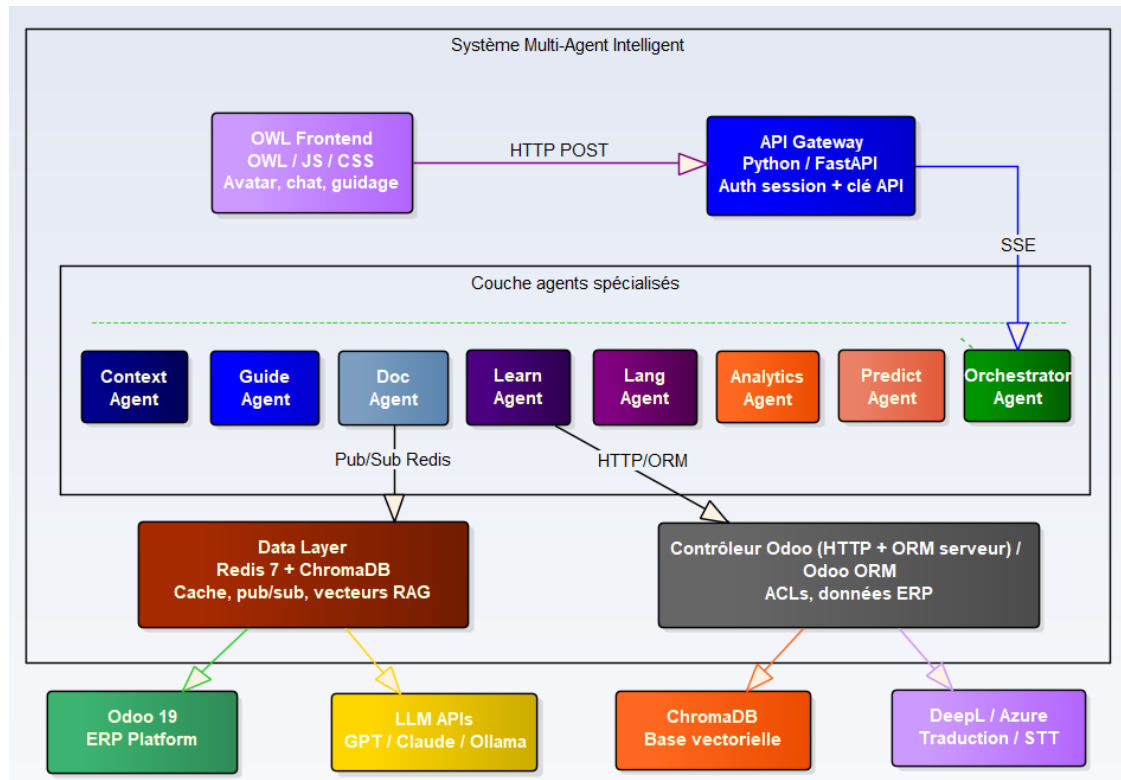


FIGURE 2 – Diagramme C4 de niveau 2 — conteneurs du système

II.3.2 Description détaillée des conteneurs

OWL Frontend — module Odoo

Technologie : Odoo Web Library (OWL) / JavaScript / SCSS

Ce conteneur constitue l'interface utilisateur du système, intégrée nativement dans Odoo 19 sous forme de module. Il est responsable de l'ensemble de la couche de présentation et d'interaction :

- **Avatar flottant interactif** : composant OWL déplaçable affiché en surimpression sur toutes les pages Odoo, animé par des animations Lottie (2D) exprimant des états émotionnels ;

- **Interface de chat** : widget de messagerie bidirectionnel avec support du texte, de la voix (Web Speech API) et du rendu Markdown des réponses ;
- **Moteur de guidage visuel** : injection CSS dynamique pour la surbrillance d'éléments, info-bulles positionnelles, flèches SVG animées ;
- **Collecteur d'événements DOM** : observateurs et gestionnaires d'événements transmettant le contexte d'interface (page active, formulaire, erreurs) vers le backend ;
- **Tableaux de bord d'administration** : vues Odoo dédiées à la supervision des agents, aux statistiques d'usage (Insights) et au suivi de l'apprentissage.

Proxy Odoo — contrôleur serveur

Technologie : contrôleur HTTP Odoo / ORM Odoo côté serveur / Python

Le navigateur n'appelle jamais directement le backend IA : un contrôleur HTTP côté serveur Odoo joue le rôle de passerelle sécurisée. Il valide la session Odoo, respecte strictement les ACLs de l'utilisateur (accès aux données métier via l'ORM), enrichit le contexte transmis, injecte la clé API côté serveur et relaie aussi bien les réponses JSON classiques que les flux SSE.

API Gateway — FastAPI

Technologie : Python / FastAPI / Uvicorn

Point d'entrée unique du backend, l'API Gateway centralise la gestion des requêtes entrantes et applique les politiques transversales :

- **Authentification et autorisation** : validation de la clé API (jeton Bearer) transmise par le proxy Odoo, en-têtes de contexte obligatoires (locataire, utilisateur, identifiant de requête) ;
- **Routage intelligent** : orientation des requêtes vers l'OrchestratorAgent selon le type de requête ;
- **Deux modes de réponse** : réponse synchrone complète (HTTP POST) ou diffusion progressive par Server-Sent Events [23] — étapes du pipeline et jetons de la réponse en direct, avec repli automatique sur le mode synchrone ;
- **Limitation de débit et fiabilité** : protection contre les abus (slowapi, quota par locataire) et résilience des appels LLM (délais stricts, circuit breaker).

Orchestrator Agent

Technologie : LangChain / LangGraph / Python

Cerveau du système, l'OrchestratorAgent reçoit les requêtes prétraitées et orchestre la collaboration des sept agents spécialisés selon un workflow défini en graphe d'états [14] :

- **Analyse d'intention** : des court-circuits déterministes (guidage explicite, QCM explicite, démonstration, conversation informelle) traitent les cas évidents sans

- appel LLM ; sinon, la requête est classifiée par le LLM (question, navigation, formation, démonstration, erreur) ;
- **Orchestration par graphe** : LangGraph définit les transitions d'état entre agents, permettant des workflows parallèles ou séquentiels ;
- **Gestion des priorités** : routage prioritaire des événements critiques (erreurs) traités en amont du flux courant.

Data Layer — Redis & ChromaDB

Technologies : Redis 7 (cache / pub-sub / streams) + ChromaDB (base vectorielle)

- **Redis** : cache des réponses documentaires (TTL configurable), sessions et progression de guidage, états de répétition espacée SM-2, flux de télémétrie, traces et métriques par agent, journal d'audit ;
- **ChromaDB** : stockage des embeddings de la documentation Odoo et du catalogue de vidéos, alimentant le pipeline RAG [7].

II.4 Diagramme C4 — Niveau 3 : composants

II.4.1 Spécification des huit agents spécialisés

La « couche agents spécialisés » se décompose en huit composants-agents, chacun possédant une responsabilité unique, des interfaces d'entrée/sortie définies et un ensemble d'outils (tools) spécifiques.

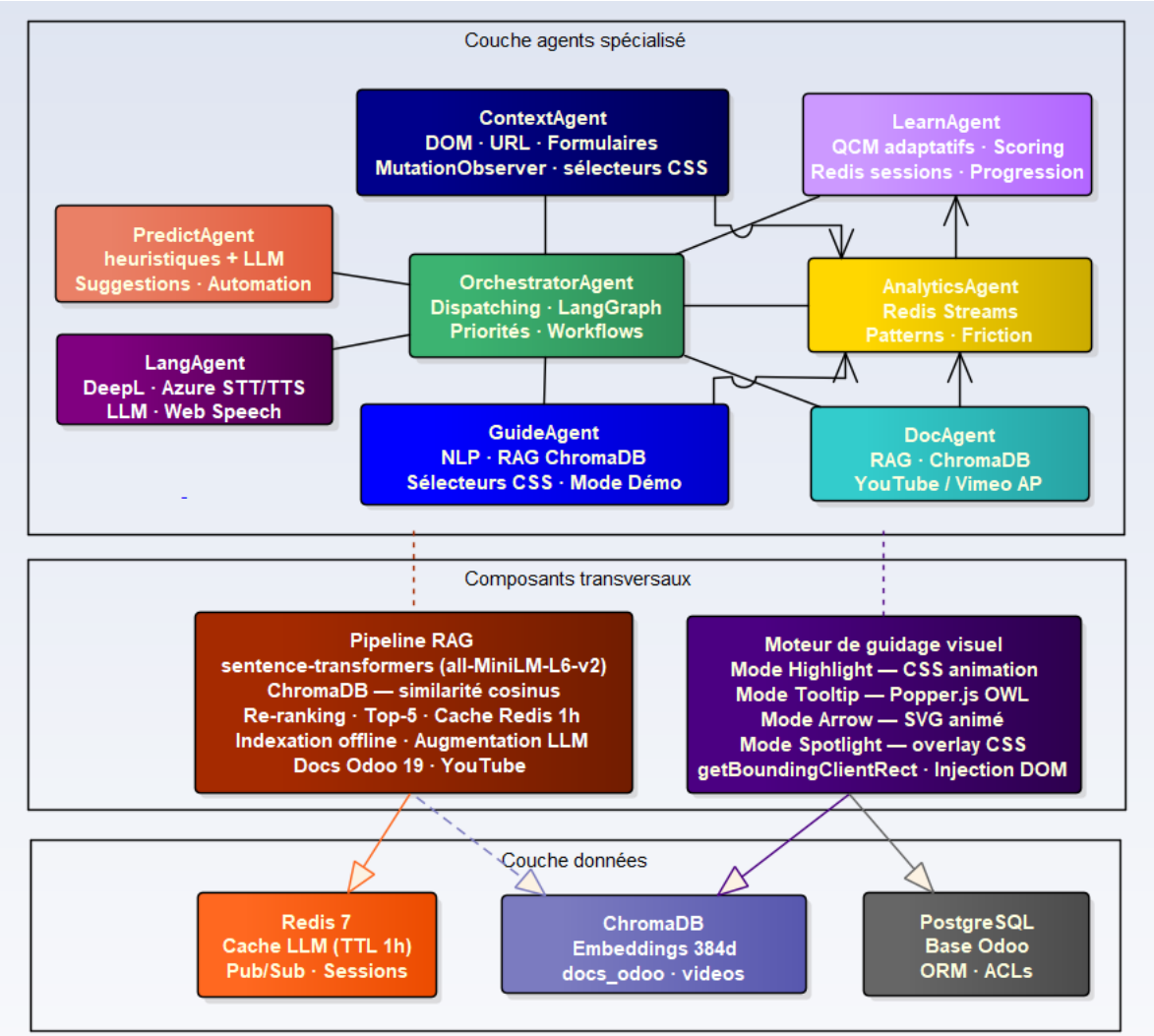


FIGURE 3 – Diagramme C4 de niveau 3 — composants de la couche agents

Agent	Rôle	Entrées	Sorties	Outils / Tech
Orchestrator	Chef d'orchestre	Requêtes utilisateur, événements	Routage, coordination du workflow	LangGraph, gates heuristiques
Context	Analyste contextuel	URL, module, état du formulaire	Contexte enrichi (JSON), niveau utilisateur	Sélecteurs CSS, historique Redis, état SM-2
Guide	Guide pas-à-pas	Question NL, contexte courant	Réponse + sélecteurs CSS, étapes de guidage	LLM + RAG ChromaDB, inférence heuristique

Agent	Rôle	Entrées	Sorties	Outils / Tech
Doc	Gestionnaire documentaire	Requête sémantique, module actif	Articles pertinents, vidéos, réponse ancrée	ChromaDB, YouTube Data API, cache Redis
Learn	Formateur adaptatif	Profil utilisateur, historique	QCM, parcours adaptatif, progression	LLM + notation serveur, SM-2, sessions Redis
Lang	Traducteur multilingue	Texte en langue quelconque	Traduction + indications vocales	LLM multilingue (DeepL optionnel), Web Speech API
Analytics	Analyste d'usage	Événements utilisateur en continu	Patterns, alertes de sous-utilisation	Redis Streams, agrégats temps réel
Predict	Anticipateur de besoins	Historique comportemental	Suggestions proactives	Heuristiques comportementales + LLM

II.4.2 Architecture interne de chaque agent

Les agents reposent sur une architecture ReAct (Reasoning + Acting) [6] implémentée via LangChain et activable par configuration ; la boucle est plafonnée à deux itérations (`max_iterations = 2`) afin de borner la latence et le coût, et par défaut les agents opèrent en appel LLM direct complété d'un outillage déterministe. La structure interne est la suivante :

Couche	Rôle et implémentation
System Prompt	Prompt système définissant la personnalité, les contraintes et le domaine de compétence de l'agent ; injecté à chaque appel LLM, défini et versionné dans un module de prompts centralisé
Memory Module	Mémoire à court terme par fenêtre glissante (dix derniers échanges) et à long terme (résumés périodiques stockés dans Redis avec TTL de 24 h)

Couche	Rôle et implémentation
Tool Registry	Ensemble d'outils (fonctions Python) que l'agent peut invoquer : recherche ChromaDB (RAG), historique et sessions Redis, résolution de sélecteurs CSS, détection d'intention, etc.
ReAct Loop	Boucle Thought → Action → Observation → Answer : l'agent raisonne sur la requête, choisit un outil, observe le résultat et itère jusqu'à formuler une réponse [6]
Output Parser	Validation et structuration de la sortie LLM : extraction des sélecteurs CSS pour le guidage, du JSON de formation ou du texte de réponse selon le type d'agent

II.4.3 Composants transversaux

Pipeline RAG (Retrieval-Augmented Generation)

Le pipeline RAG [7] est partagé entre le DocAgent et le GuideAgent. Il se décompose en trois sous-composants :

- **Indexation (hors ligne)** : ingestion de la documentation Odoo 19 (fichiers RST/Markdown/HTML), découpage en segments de 512 jetons avec recouvrement de 64 jetons, vectorisation via sentence-transformers (all-MiniLM-L6-v2, dimension 384) [8], stockage dans ChromaDB ;
- **Récupération (en ligne)** : recherche par similarité cosinus dans ChromaDB, réordonnancement par un score combinant similarité sémantique, fraîcheur du document et pertinence du module, complété d'un renforcement lexical, d'un seuil plancher de pertinence et d'une déduplication par source ; retour des cinq meilleurs segments ;
- **Augmentation** : injection des segments dans le prompt LLM avec contrainte de citation des sources et boucle de validation prévenant les hallucinations.

Moteur de guidage visuel

Ce composant, piloté par le GuideAgent côté backend et par le frontend OWL côté client, implémente trois modes de guidage :

- **Mode Highlight** : injection d'une classe CSS de surbrillance sur l'élément cible, animation de pulsation, bordure colorée ;
- **Mode Tooltip** : affichage d'une bulle d'aide positionnée dynamiquement ;
- **Mode Arrow** : flèche SVG animée pointant vers l'élément cible, calculée à partir de ses coordonnées (getBoundingClientRect) ;

- **Mode Spotlight** : voile semi-transparent sur toute la page sauf la région cible, forçant l'attention visuelle.

Canal de progression SSE

Composant transversal de diffusion en temps réel [23] : les nœuds du graphe d'orchestration émettent des événements d'étape (début/fin de chaque agent) et les appels LLM « dernier maillon » diffusent leurs jetons. Ces événements transitent par une file locale au contexte de la requête, drainée par la réponse HTTP en flux text/event-stream, ce qui découple totalement le pipeline synchrone de la couche de diffusion.

II.5 Flux de données

II.5.1 Flux principal — question utilisateur

Ce flux décrit le traitement complet d'une question posée en langage naturel, depuis la saisie jusqu'à l'affichage de la réponse avec guidage visuel.

N°	Acteur / composant	Action — Flux 1 : question utilisateur → réponse avec guidage visuel
1	Utilisateur	Saisit une question en langage naturel dans le chat de l'avatar ou par commande vocale (STT via Web Speech API)
2	OWL Frontend	Collecte le contexte courant (URL, module Odoo actif, vue, formulaire en cours) et le joint à la requête, acheminée par le proxy Odoo (RPC JSON ou flux SSE)
3	Proxy Odoo	Valide la session et les ACLs, enrichit le contexte métier via l'ORM serveur, injecte la clé API puis transmet au backend
4	API Gateway	Vérifie le jeton Bearer et les en-têtes de contexte, applique la limitation de débit, transmet à l'OrchestratorAgent ; en mode flux, émet immédiatement les premières trames SSE
5	OrchestratorAgent	Applique d'abord les court-circuits heuristiques (guidage explicite, QCM, démonstration, conversation informelle) ; sinon classe l'intention via LLM et détermine le workflow d'agents à activer

N°	Acteur / composant	Action — Flux 1 : question utilisateur → réponse avec guidage visuel
6	ContextAgent	Enrichit le contexte : position exacte dans l'interface, profil de l'utilisateur (niveau dérivé de l'état SM-2, langue, historique) depuis Redis
7	GuideAgent DocAgent	S'exécutent en parallèle : le GuideAgent génère la réponse et les sélecteurs CSS des éléments à mettre en évidence ; le DocAgent interroge ChromaDB (RAG) et produit une réponse documentaire ancrée dans les sources
8	OrchestratorAgent	Synthétise les contributions (appel LLM de synthèse uniquement si les deux agents ont contribué) puis calcule un score de confiance ; une unique boucle de réessai est déclenchée si les deux agents ont échoué avec une confiance faible
9	LangAgent	Traduit la réponse dans la langue préférée de l'utilisateur si elle diffère de la langue de génération
10	API Gateway	Renvoie la réponse complète (trame finale SSE ou réponse HTTP), faisant foi sur l'aperçu diffusé
11	OWL Frontend	Affiche la réponse (rendu Markdown), active le guidage visuel (surbrillance, info-bulle, flèche) selon les sélecteurs reçus
12	AnalyticsAgent	Enregistre l'événement (type de question, module, latence) dans un flux Redis pour l'analyse des patterns

II.5.2 Flux de formation adaptative

N°	Acteur / composant	Action — Flux 2 : déclenchement et déroulement d'une session de formation
1	Utilisateur / PredictAgent	La formation est demandée explicitement (« forme-moi sur les ventes ») ou suggérée proactivement à partir des frictions détectées
2	OrchestratorAgent	Le court-circuit « formation explicite » active directement le LearnAgent avec le profil utilisateur et le module cible, sans appels LLM intermédiaires

N°	Acteur / composant	Action — Flux 2 : déclenchement et déroulement d'une session de formation
3	LearnAgent	Génère un QCM d'évaluation à difficulté adaptée au niveau de l'utilisateur (dérivé de son état de répétition espacée SM-2 [11]), via LLM
4	OWL Frontend	Affiche le QCM dans une boîte de dialogue dédiée et collecte les réponses
5	LearnAgent	Note les réponses côté serveur (les corrigés ne quittent jamais le backend), met à jour l'état SM-2 et l'historique dans Redis
6	LearnAgent	Génère un parcours de formation adaptatif ancré dans la documentation réelle récupérée via le pipeline RAG
7	GuideAgent + DocAgent	Produisent les étapes du parcours : chaque étape combine explication textuelle, guidage visuel et ressource documentaire ou vidéo pertinente
8	OWL Frontend	Affiche le parcours avec barre de progression, active le guidage visuel à chaque étape et propose des validations intermédiaires
9	Redis	Persiste la progression (module, étape, score) pour reprise après déconnexion ; les tableaux de bord Odoo (Mon niveau, Historique QCM, Succès, Statistiques) restituent ces données

II.5.3 Flux de mode démonstration

N°	Acteur / composant	Action — Flux 3 : exécution du mode démonstration (avatar acteur)
1	Utilisateur	Demande : « Montre-moi comment créer une facture client dans Odoo »
2	OrchestratorAgent	Le court-circuit « démonstration » identifie la requête et active le GuideAgent en mode démonstration
3	GuideAgent	Décompose la tâche en sous-actions atomiques (naviguer → créer → saisir → confirmer) à partir de sa base de connaissances Odoo

N°	Acteur / composant	Action — Flux 3 : exécution du mode démonstration (avatar acteur)
4	GuideAgent	Pour chaque action : génère le sélecteur CSS de l'élément cible, la narration explicative et le type d'interaction (clic, saisie, défilement)
5	OWL Frontend	Exécute la séquence : déplace le curseur-avatar vers l'élément, met l'élément en surbrillance, affiche la narration
6	OWL Frontend	La démonstration se déroule sur l'interface réelle en lecture seule : le contrat de données n'autorise que des actions d'affichage et de navigation, et une confirmation explicite de l'utilisateur est requise avant chaque étape
7	LangAgent	Assure la narration dans la langue préférée de l'utilisateur, avec synthèse vocale optionnelle via la Web Speech API

II.5.4 Flux de détection et de résolution d'erreur

N°	Acteur / composant	Action — Flux 4 : détection contextuelle et résolution d'erreur
1	OWL Frontend	Détecte l'apparition d'un message d'erreur dans l'interface Odoo (notifications d'erreur, alertes de validation, exceptions réseau)
2	ContextAgent	Capture le message d'erreur, l'URL courante, l'état du formulaire et le contexte de navigation immédiatement antérieur
3	OrchestratorAgent	Interruption à haute priorité : active le GuideAgent en mode résolution d'erreur
4	GuideAgent	Interroge ChromaDB avec le message d'erreur pour retrouver des cas documentés similaires et consulte l'historique Redis des erreurs de l'utilisateur
5	GuideAgent	Génère via LLM une explication claire de la cause et des étapes de résolution, avec les sélecteurs CSS des champs à corriger
6	PredictAgent	Vérifie si l'erreur s'inscrit dans un schéma récurrent et formule des recommandations préventives

N°	Acteur / composant	Action — Flux 4 : détection contextuelle et résolution d'erreur
7	OWL Frontend	Affiche l'explication dans une notification prioritaire de l'avatar et met en évidence les champs problématiques
8	AnalyticsAgent	Enregistre l'erreur dans le profil de friction de l'utilisateur et met à jour les métriques globales

II.5.5 Flux RAG — pipeline de récupération documentaire

N°	Acteur / composant	Action — Flux 5 : pipeline RAG (Retrieval-Augmented Generation) [7]
1	DocAgent	Reçoit une requête documentaire avec le contexte sémantique (module actif, question, historique récent)
2	Redis Cache	Vérifie si la requête (empreinte SHA-256 de la question, de l'intention et du module) est en cache ; un succès renvoie immédiatement les documents et la réponse ancrée (TTL : 1 h pour les documents, 24 h pour les vidéos)
3	DocAgent	Translittère les termes français vers l'anglais via un glossaire déterministe (le corpus est anglophone) puis génère l'embedding de la requête via sentence-transformers (all-MiniLM-L6-v2, dimension 384) [8]
4	ChromaDB	Recherche par similarité cosinus dans les collections du corpus Odoo et du catalogue de vidéos, complétée par la recherche YouTube en direct
5	DocAgent	Réordonnance les candidats : score combiné ($0,7 \times \text{similarité} + 0,2 \times \text{fraîcheur} + 0,1 \times \text{pertinence module}$) renforcé d'un boost lexical sur le titre ; les scores sont normalisés dans $[0, 1]$, les candidats sous le seuil plancher sont écartés, les doublons d'URL supprimés et la diversité des sources garantie ; les cinq meilleurs sont retenus
6	DocAgent + LLM	Construit le prompt augmenté — prompt système + cinq meilleurs segments avec leurs sources + question — et appelle le LLM avec contrainte de citation des sources

N°	Acteur / composant	Action — Flux 5 : pipeline RAG (Retrieval-Augmented Generation) [7]
7	DocAgent	Valide la réponse (présence de citations, fidélité aux sources, pertinence au module); un score heuristique inférieur à 0,7 entraîne le rejet et une recommandation de reformulation
8	Redis Cache	Stocke la réponse validée et les documents associés en cache
9	GuideAgent / LearnAgent	Reçoivent la réponse documentaire enrichie et l'intègrent dans leur propre réponse à l'utilisateur

II.6 Communication inter-agents

II.6.1 Protocoles de communication

La coordination des agents repose sur deux mécanismes complémentaires :

- **État partagé du graphe LangGraph (chemin synchrone)** : le flux critique qui produit la réponse circule dans l'état partagé du graphe d'orchestration, ce qui garantit l'ordre et la cohérence sans latence réseau supplémentaire ;
- **Redis (chemin asynchrone)** [22] : les échanges non bloquants — télémétrie vers l'AnalyticsAgent, retours d'adaptation vers le LearnAgent — transitent par Redis : pub/sub pour les notifications et Redis Streams, avec accusés de réception, pour les messages critiques.

II.6.2 Matrice des échanges

Légende : $\rightarrow$ (émission) / $\leftarrow$ (réception) / $\leftrightarrow$ (bidirectionnel) / $-$ (sans objet) / case vide (pas de communication directe)

Agent \ Cible	Orch.	Context	Guide	Doc	Learn	Lang	Analyt.	Predict
Orchestrator	—	$\leftrightarrow$	$\leftrightarrow$	$\leftrightarrow$	$\leftrightarrow$	$\leftrightarrow$	$\leftrightarrow$	$\leftrightarrow$
Context	$\leftrightarrow$	—					$\rightarrow$	
Guide	$\leftrightarrow$		—				$\rightarrow$	
Doc	$\leftrightarrow$			—			$\rightarrow$	
Learn	$\leftrightarrow$				—		$\leftarrow$	

Agent \ Cible	Orch.	Context	Guide	Doc	Learn	Lang	Analyt.	Predict
Lang	↔					—		
Analytics	↔	←	←	←	→		—	
Predict	↔							—

II.6.3 Protocole de message standard

Tous les messages inter-agents transitant par Redis respectent un format JSON standardisé :

Champ	Type	Description
message_id	UUID v4	Identifiant unique du message pour déduplication et traçabilité
source_agent	énumération	Nom de l'agent émetteur (orchestrator, context, guide, doc, learn, lang, analytics, predict)
target_agent	énumération broadcast	Agent(s) destinataire(s) ; « broadcast » pour tous les agents
message_type	énumération	Type : TASK RESULT EVENT ERROR HEARTBEAT
priority	entier [0–10]	Priorité : 10 = urgence maximale (erreur critique), 0 = tâche de fond
session_id	chaîne	Identifiant de la session utilisateur Odoo, lien avec le contexte Redis
payload	objet JSON	Données spécifiques au message (question, contexte, résultat RAG, etc.)
timestamp	ISO 8601	Horodatage UTC de création du message
ttl	entier (secondes)	Durée de vie du message en file avant expiration (défaut : 300 s)

II.7 Choix technologiques et stack complète

II.7.1 Vue d'ensemble de la stack

Frontend	Backend	IA / LLM	Data	DevOps
OWL Framework (Odoo 19) Lottie (avatar 2D) Animations CSS (spotlight) Web Speech API (STT/TTS) SSE via fetch + ReadableS- tream	Python 3.11 FastAPI (async) LangChain / LangGraph Uvicorn (ASGI) Pydantic v2 slowapi (rate limiting)	NVIDIA NIM (défaut) OpenAI / Claude / Ollama (configurables) Circuit breaker LLM ChromaDB (vecteurs) Sentence- Transformers	Redis 7 (cache, pub/sub, streams) PostgreSQL (base Odoo) ChromaDB (RAG) YouTube Data API v3 DeepL API (optionnel)	Docker / Docker Compose Kubernetes (manifestes) GitHub Actions (CI/CD) Nginx (reverse proxy) Prometheus / Grafana pytest, Playwright, ruff

II.7.2 Justification des choix clés

FastAPI vs Flask/Django

FastAPI [17] a été préféré pour ses performances asynchrones natives (async/await), sa génération automatique de documentation OpenAPI et son typage strict via Pydantic — essentiel pour valider les formats de messages inter-agents. Les comparatifs indépendants TechEmpower [24] le classent parmi les frameworks web Python les plus performants pour les charges dominées par les entrées/sorties, caractéristiques de ce projet.

LangChain + LangGraph vs AutoGen / CrewAI

LangChain [13] offre l'écosystème le plus mature pour l'intégration de LLM avec des outils personnalisés. LangGraph [14] y ajoute l'orchestration par graphe d'états, permettant des workflows non linéaires (boucles, conditions, parallélisme) impossibles avec les frameworks linéaires. AutoGen [16] a été écarté car son modèle conversationnel convient moins aux agents orientés tâche, et CrewAI [15] restait moins flexible pour le contrôle fin des transitions d'état.

ChromaDB vs Pinecone

ChromaDB [18] a été choisi pour le développement et le déploiement initial grâce à son installation locale sans dépendance cloud, alignée avec les contraintes de coûts du cahier des charges, et à son mode serveur autorisant plusieurs réplicas du backend. Une migration vers une base vectorielle managée reste possible pour des déploiements dépassant le million de vecteurs.

OWL Framework (Odoo Web Library)

L'utilisation du framework OWL natif d'Odoo 19 [20] est à la fois une contrainte et un avantage : elle garantit une compatibilité parfaite avec le système de composants Odoo, l'accès aux services natifs du client web (dont l'appel RPC sécurisé) et l'intégration dans le cycle de mise à jour des modules.

Redis comme bus de données

Redis [22] centralise quatre fonctions critiques : (1) cache des réponses documentaires pour réduire les coûts d'API, (2) pub/sub et streams pour la communication asynchrone inter-agents, (3) persistance des sessions, de la progression de guidage et des états SM-2, (4) journal d'audit et métriques par agent. Sa latence sous la milliseconde en fait le choix idéal pour une communication en temps réel, et l'externalisation de l'état y rend le backend totalement sans état.

Server-Sent Events pour la diffusion progressive

Le standard SSE [23] a été retenu plutôt que WebSocket pour la diffusion de la réponse : le flux est unidirectionnel (serveur → client), s'appuie sur HTTP simple (compatible avec le proxy Odoo et les reverse proxies), et se replie naturellement sur la réponse synchrone en cas d'indisponibilité.

II.7.3 Stratégie de déploiement

L'architecture est déployée via Docker Compose ; l'instance Odoo 19 s'exécute comme service natif et dialogue avec le backend conteneurisé via le proxy HTTP. Les services conteneurisés sont :

Service Docker	Port	Description
mas-backend	8000	FastAPI + Uvicorn (ASGI) — API, orchestrateur LangGraph et agents spécialisés
mas-redis	6379	Redis 7 — cache, pub/sub, streams, sessions
mas-chromadb	8001	ChromaDB en mode serveur — base vectorielle RAG
mas-indexer	interne	Service d'ingestion et d'indexation de la documentation
mas-nginx	80 / 443	Reverse proxy, terminaison TLS
mas-prometheus	9090	Collecte des métriques
mas-grafana	3000	Visualisation des métriques

Pour la production multi-instances, des manifestes Kubernetes complets accompagnent le projet : déploiement du backend avec sondes de vivacité et de disponibilité, autoscaler horizontal (de deux à six réplicas), Ingress TLS, ConfigMap et Secrets. Cette trajectoire est rendue possible par le caractère sans état du backend.

Conclusion

Cette phase de conception a permis de définir clairement l'architecture du système multi-agent en s'appuyant sur trois niveaux complémentaires du modèle C4 : le contexte global, les conteneurs et les composants. Cette organisation offre une vision à la fois générale et détaillée du système, en identifiant les utilisateurs, les systèmes externes ainsi que les différents agents et leurs rôles.

L'architecture adoptée repose sur une séparation nette des responsabilités (interface, proxy, API, agents, données), ce qui facilite la maintenance, l'évolutivité et la compréhension globale de la solution. Les cinq flux de données couvrent les cas d'utilisation essentiels — interactions utilisateur, formation, démonstration, gestion des erreurs et récupération documentaire — et la diffusion progressive par SSE répond à l'exigence de latence perçue.

Enfin, la communication entre les agents est standardisée : le chemin critique transite par l'état partagé du graphe d'orchestration tandis que les échanges asynchrones sont assurés par Redis, garantissant des interactions fiables, rapides et cohérentes. Le chapitre suivant décrit la mise en œuvre concrète de cette architecture.

Chapitre III : Réalisation

Après la phase de conception, ce chapitre décrit la mise en œuvre concrète du Système Multi-Agent Intelligent, commercialisé sous le nom **Maxware AI Assistant** (module Odoo « Maxware », version 19.0.2.x). Il présente l'environnement technique retenu, l'implémentation du backend multi-agent puis celle du module Odoo, la sécurité de bout en bout et enfin la stratégie de déploiement. L'objectif est de montrer comment les choix d'architecture se traduisent en code fonctionnel et exécutable.

III.1 Environnement de développement et outils

Le projet repose sur deux briques principales : un backend Python conteneurisé exposant l'intelligence multi-agent (dépôt « backend_ai »), et un module Odoo natif assurant l'interface utilisateur (« mas_assistant_frontend »). Le tableau suivant récapitule les outils retenus et leur rôle.

Brique	Technologie	Rôle
Backend	Python 3.11, FastAPI, Uvicorn	API asynchrone et orchestration des agents
Orchestration	LangChain, LangGraph	Graphe d'états et outillage des agents
LLM	NVIDIA NIM (défaut) ; OpenAI / Claude / Ollama	Génération de réponses via une couche multi-fournisseur
RAG	ChromaDB (mode serveur), sentence-transformers	Recherche sémantique sur la documentation (corpus 4 405 segments)
Cache / bus	Redis 7	Cache, sessions, pub/sub, streams, audit
Frontend	Odoo 19, OWL, JavaScript, SCSS	Avatar, chat, guidage visuel, tableaux de bord
Conteneurisation	Docker, Docker Compose, Kubernetes	Isolation, reproductibilité, montée en charge
Qualité et tests	pytest, Playwright, ruff, GitHub Actions	Tests unitaires et de bout en bout, lint, intégration continue

III.2 Implémentation du backend multi-agent

III.2.1 Orchestrateur (LangGraph)

L'OrchestratorAgent est implémenté sous la forme d'un graphe d'états LangGraph [14] composé des nœuds suivants : classification, enrichissement du contexte, exécution parallèle des agents Guide et Doc, formation, analyse, finalisation, vérification de réessai, prédiction et traduction. L'état partagé du graphe transporte la requête, le contexte enrichi et les contributions de chaque agent jusqu'à la réponse finale.

Court-circuits heuristiques de classification. Avant tout appel au LLM, des portes déterministes interceptent les cas évidents : poursuite d'un guidage en cours, demande de démonstration, guidage explicite (« guide-moi pour... »), formation explicite (« QCM sur... ») et conversation informelle (salutations, heure, date...). Chaque porte économise deux à quatre appels LLM et de trois à dix secondes de latence, et active directement le sous-ensemble d'agents pertinent.

Synthèse et fiabilité. La synthèse par LLM n'est déclenchée que lorsque le GuideAgent et le DocAgent ont tous deux contribué ; dans le cas contraire, une réponse documentaire structurée est produite sans appel supplémentaire. Une arête conditionnelle autorise une unique boucle de réessai, déclenchée seulement si les deux agents principaux ont échoué avec une confiance inférieure à 0,4. Le score de confiance, dérivé des traces d'exécution des agents, est borné entre 0,1 et 0,99, et l'ensemble du pipeline est protégé par un délai d'expiration strict garantissant la réactivité même en cas de lenteur du fournisseur LLM.

III.2.2 Les agents spécialisés

Chaque agent possède une responsabilité unique et expose une interface d'entrée/sortie claire. Le ContextAgent enrichit le contexte courant (module, vue, intention, niveau de l'utilisateur dérivé de son état de répétition espacée). Le GuideAgent produit les réponses et les séquences de guidage visuel (sélecteurs CSS, info-bulles, flèches) selon trois modes — guidage, démonstration et résolution d'erreur — et dispose d'une inférence heuristique de guidage qui évite l'appel LLM pour les parcours connus. Le DocAgent alimente le pipeline RAG et la recherche de vidéos. Le LearnAgent génère les QCM et les parcours de formation et assure la notation côté serveur. Le LangAgent détecte la langue et traduit les réponses. L'AnalyticsAgent consomme le flux de télémétrie et calcule les métriques par agent. Le PredictAgent anticipe les besoins à partir d'heuristiques comportementales complétées par le LLM.

III.2.3 Couche LLM multi-fournisseur et résilience

Pour ne pas figer le système sur un unique fournisseur, l'accès au LLM passe par une abstraction commune (un protocole exposant les méthodes d'appel synchrone, d'appel en flux et de contrôle de santé). Le fournisseur par défaut est NVIDIA NIM, retenu pendant le développement pour son programme d'accès développeur gratuit ; un fournisseur générique compatible avec l'API OpenAI couvre, par simple configuration, OpenAI, Claude et Ollama. Le fournisseur et le modèle actifs sont sélectionnés par variables d'environnement, sans modification du code.

Circuit breaker. Tous les appels (synchrone et en flux) traversent un disjoncteur logiciel : après cinq échecs consécutifs, le circuit s'ouvre pendant trente secondes puis passe en état semi-ouvert avec une unique sonde. En cas de panne du fournisseur, les requêtes échouent ainsi en quelques millisecondes au lieu de subir le délai d'expiration complet, et les replis déterministes des agents (classification heuristique, réponses structurées) absorbent l'indisponibilité sans interrompre le service.

III.2.4 Pipeline RAG

Le pipeline de récupération documentaire [7] se décompose en trois temps. **Hors ligne**, la documentation Odoo [19] est découpée en segments de 512 jetons avec un recouvrement de 64 jetons, vectorisée par le modèle all-MiniLM-L6-v2 (dimension 384) [8] et stockée dans ChromaDB ; le corpus indexé compte 4 405 segments. **En ligne**, la requête — préalablement translittérée du français vers l'anglais par un glossaire déterministe, le corpus étant anglophone — est vectorisée puis comparée par similarité cosinus ; les candidats sont réordonnés par un score combinant similarité sémantique (0,7), fraîcheur du document (0,2) et pertinence du module (0,1), renforcé d'un boost lexical sur le recouvrement des titres. Les scores sont normalisés dans l'intervalle [0, 1], un seuil plancher écarte les correspondances quasi aléatoires, les doublons d'URL sont supprimés et la diversité des sources est garantie (au plus deux documents par source, cinq documents retenus). **Enfin**, ces segments sont injectés dans l'invite du LLM avec une contrainte de citation des sources (véritable « retrieve-then-generate ») ; chaque réponse est ensuite contrôlée par une validation heuristique — score initialisé à 0,8, pénalisé en l'absence de sources citées, pour une longueur inadéquate ou en présence de marqueurs d'incertitude — et toute réponse sous le seuil de 0,7 est marquée à reformuler, le score alimentant la confiance globale. Les résultats — documents et réponse ancrée — sont mis en cache (une heure pour les documents, vingt-quatre heures pour les vidéos), ramenant les requêtes répétées à un temps quasi nul.

III.2.5 Communication inter-agents

La coordination repose sur les deux mécanismes définis en conception. Le chemin synchrone critique — celui qui produit la réponse — est assuré par l'état partagé du graphe LangGraph, ce qui garantit l'ordre et la cohérence sans latence réseau supplémentaire. Les échanges asynchrones (télémétrie vers l'AnalyticsAgent, retours d'adaptation vers le LearnAgent) transitent par Redis : pub/sub pour les notifications et Redis Streams, avec accusés de réception, pour les messages critiques. Les messages respectent le format standardisé décrit au chapitre II (identifiant, agents source et cible, type, priorité, charge utile, horodatage, durée de vie).

III.2.6 Streaming SSE de bout en bout

Le pipeline d'orchestration étant synchrone, la diffusion progressive repose sur un **canal de progression local au contexte de la requête** [23] : l'endpoint de flux installe une file en mémoire dans une copie du contexte d'exécution et lance le pipeline dans un fil dédié. Les nœuds du graphe émettent des événements d'étape (début et fin de chaque agent), et les appels LLM « dernier maillon » — synthèse, réécriture de guidage, augmentation documentaire, conversation informelle — diffusent leurs jetons au fil de la génération.

La réponse HTTP draine cette file sous forme de trames « text/event-stream » typées : ouverture (identifiant de requête), étape (nom et statut), jeton (delta de texte), trame finale (réponse complète faisant foi) et erreur. Deux garde-fous structurent ce contrat : les appels LLM produisant des sorties structurées (classification, génération de QCM) ne sont jamais diffusés vers le client, et le texte diffusé n'est qu'un aperçu — la traduction peut encore modifier la réponse — seule la trame finale faisant foi. En l'absence de flux actif, les émissions sont des opérations vides : le chemin synchrone reste strictement inchangé.

Le proxy Odoo relaie les trames telles quelles et normalise la trame finale avec exactement le même traitement que la réponse classique ; côté client, le widget consomme le flux, affiche la véritable étape du pipeline en cours, peint les jetons dans une bulle « brouillon », puis remplace celle-ci par le rendu définitif à la réception de la trame finale. Tout échec du flux déclenche un repli transparent sur la requête synchrone.

III.2.7 Contrat d'API du backend

Le backend expose une API REST versionnée dont les principales routes sont récapitulées ci-dessous ; toutes les routes de données exigent le jeton Bearer et les en-têtes de contexte,

tandis que les sondes de santé restent ouvertes.

Route	Méthode	Rôle
/api/v1/assist	POST	Traitement complet d'une requête utilisateur (réponse synchrone)
/api/v1/assist/stream	POST	Même traitement avec diffusion progressive SSE
/api/v1/learn/answer, /learn/profile, /learn/journey, /learn/journey/next	POST / GET	Notation des QCM, profil d'apprentissage, génération et avancement des parcours
/api/v1/predict/suggestions	POST	Suggestions proactives
/api/v1/events	POST	Ingestion de la télémétrie d'usage
/api/v1/analytics/summary	GET	Agrégats d'usage pour les tableaux de bord
/api/v1/agents/status	GET	État et métriques de chaque agent
/api/v1/feedback	POST	Retour utilisateur sur les réponses
/api/v1/session/start	POST	Initialisation de session
/health, /health/live, /health/ready	GET	Sondes de santé (vivacité et disponibilité)
/api/v1/admin/reload-settings	POST	Rechargement de la configuration (clé d'administration dédiée)

III.3 Implémentation du module Odoo (frontend OWL)

III.3.1 Avatar et communication

L'interface se présente sous la forme d'un avatar flottant déplaçable, implémenté comme composant OWL [20] affiché en surimpression sur toutes les pages Odoo. L'avatar est animé par des animations Lottie locales — repos, réflexion, parole, célébration — pour exprimer des états émotionnels, avec repli automatique sur une icône en cas d'indisponibilité de la bibliothèque d'animation. La communication s'effectue par chat textuel (avec rendu Markdown sécurisé des réponses) et par la voix grâce à la Web Speech API du navigateur (reconnaissance et synthèse vocales). L'état du service est réactif : le widget s'abonne à l'état partagé du service OWL, garantissant le re-rendu automatique de l'interface à chaque mutation.

La capture suivante situe l'ensemble : l'avatar flotte au-dessus de l'accueil d'Odoo, prêt

à être sollicité, sans jamais bloquer le travail en cours.

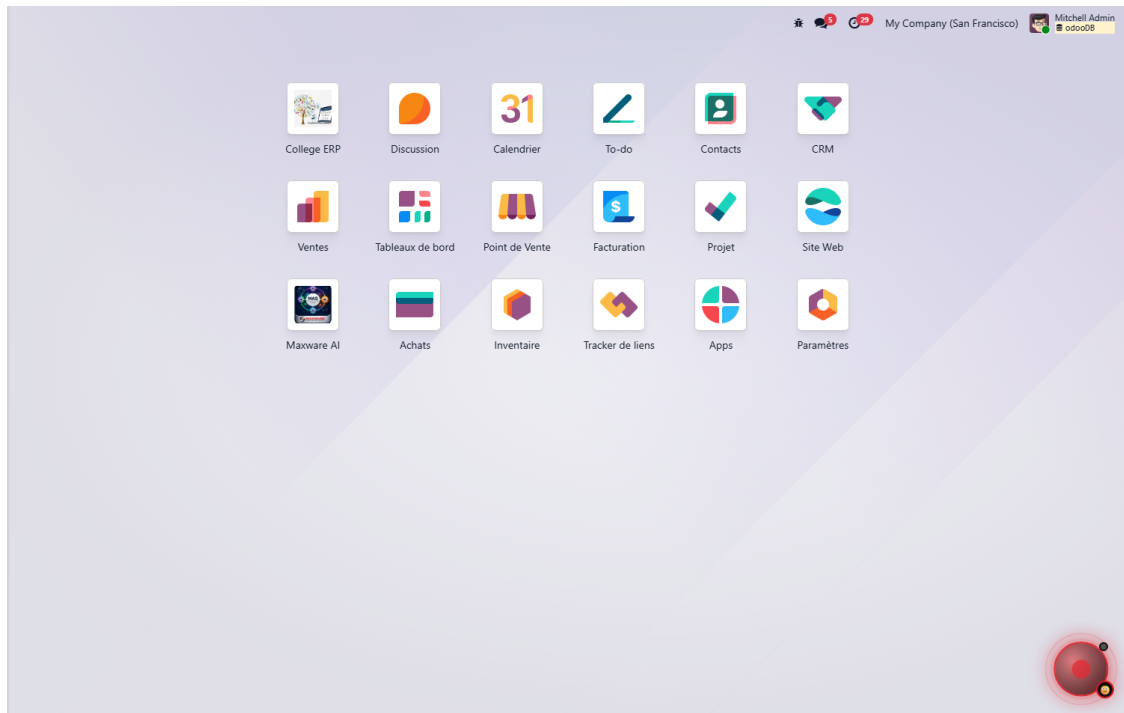


FIGURE 4 – Avatar flottant Maxware superposé à l'accueil d'Odoo 19, panneau fermé

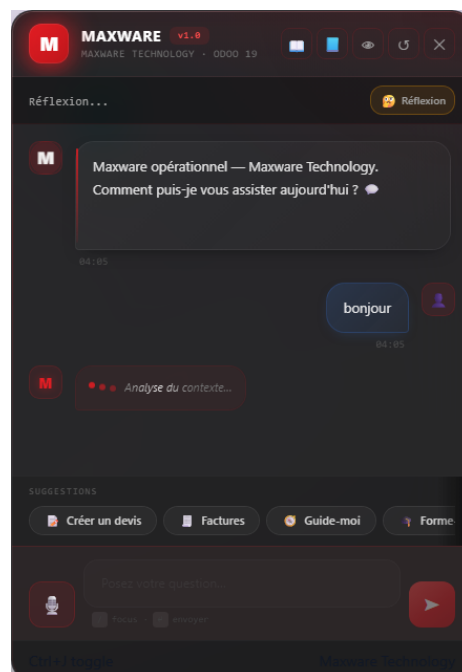


FIGURE 5 – Interface de chat de l'assistant Maxware (widget OWL flottant)

La conversation est plus riche qu'un simple échange de texte. Les réponses sont rendues en Markdown — listes, mise en gras, titres — et chaque message porte des badges de

traçabilité indiquant l'agent et le fournisseur LLM qui l'ont produit.

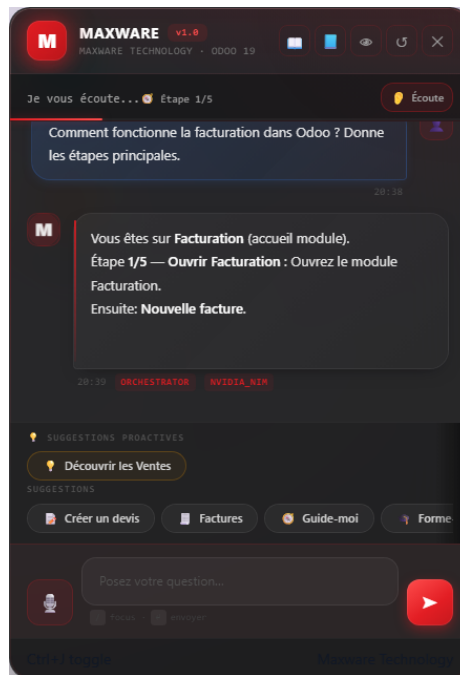


FIGURE 6 – Réponse rendue en Markdown, avec badges de traçabilité (agent et fournisseur LLM)

La voix complète le clavier : la Web Speech API assure la dictée comme la lecture des réponses, avec un état d'écoute animé directement dans la barre de saisie.

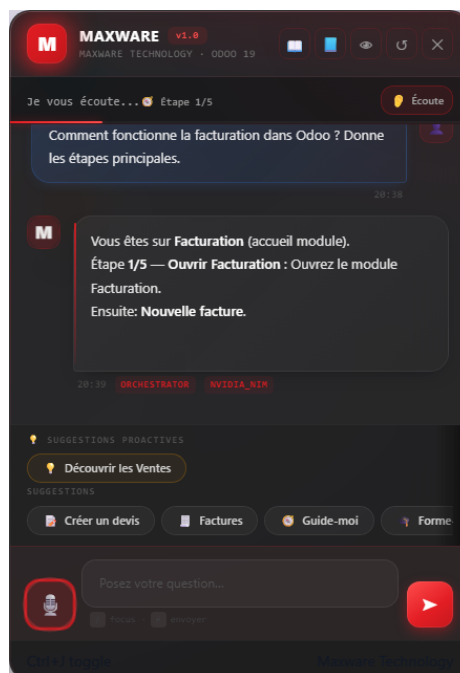


FIGURE 7 – Saisie vocale active : micro en écoute et retour visuel « Je vous écoute... »

Le multilinguisme est traité de bout en bout par le LangAgent : la langue du message est détectée et la réponse — documentation comprise — est produite dans cette langue, comme ici pour une question posée en anglais.

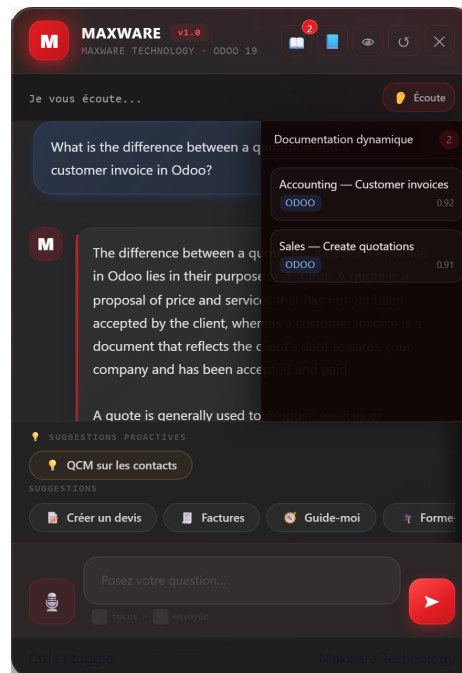


FIGURE 8 – Question posée en anglais : détection de la langue et réponse traduite par le LangAgent

L'assistant ne se contente pas de répondre : le PredictAgent alimente un bandeau de suggestions proactives, calculées à partir des habitudes de navigation, que l'utilisateur reste libre d'ignorer.

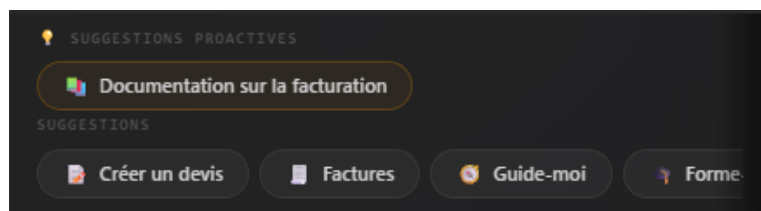


FIGURE 9 – Bandeau de suggestions proactives alimenté par le PredictAgent

Lorsque la réponse s'appuie sur la documentation, le panneau « Documentation dynamique » liste les sources retenues par le pipeline RAG avec leur score de pertinence, et les tutoriels vidéo recommandés s'affichent aux côtés des pages officielles : l'utilisateur voit d'où vient chaque affirmation.

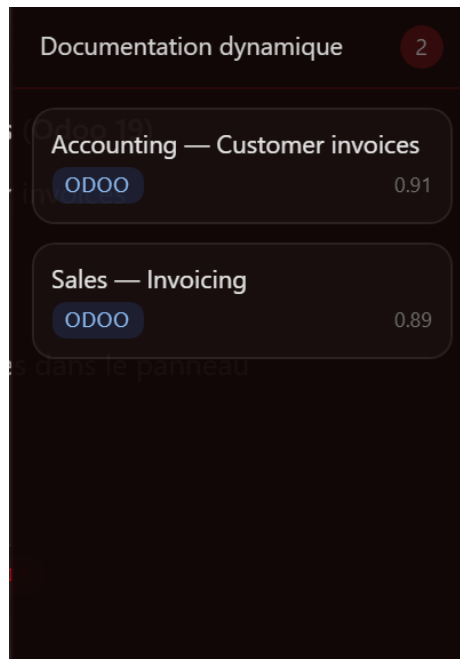


FIGURE 10 – Panneau « Documentation dynamique » : sources classées avec leur score de pertinence

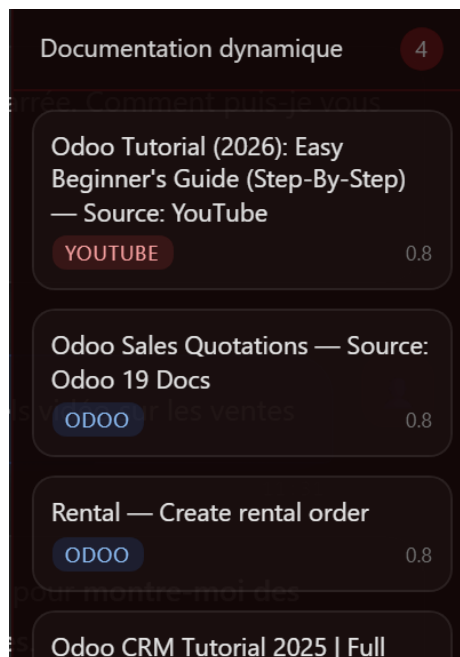


FIGURE 11 – Tutoriels vidéo YouTube recommandés aux côtés de la documentation Odoo

III.3.2 Moteur de guidage visuel

Le moteur de guidage met en évidence les éléments pertinents de l'interface selon trois modes : surbrillance par injection d'une classe CSS animée, info-bulle positionnée

dynamiquement et flèche SVG calculée à partir des coordonnées de l'élément cible. La résolution des cibles est robuste (sélection du plus grand élément visible, resserrement du cadre aux contrôles réels) et l'avancement du guidage est piloté par l'action réelle de l'utilisateur — clic sur la cible ou changement d'écran — avec démontage systématique des superpositions dès qu'une réponse sans guidage est affichée. Les sélecteurs CSS étant fournis par le GuideAgent, la logique de guidage reste découplée de la structure précise du DOM.

Ancrage par introspection (couverture universelle). Pour que ces sélecteurs soient exacts sur n'importe quel module — y compris les modules développés en interne par l'entreprise — le GuideAgent ne se limite pas à des sélecteurs génériques : il interroge la structure réelle du modèle cible. Un point d'accès dédié du module Odoo (POST /maxware/introspect, dont l'accès respecte les droits de l'utilisateur connecté) expose, à partir de `fields_get` et de l'analyse de l'arch retournée par `get_view`, les champs réellement présents (libellé, type, caractère obligatoire, visibilité) et les boutons de la barre d'état (méthodes de workflow). Le backend en dérive des sélecteurs DOM exacts du client OWL — par exemple « `.o_field_widget[name='partner_id']` » pour le champ Client, ou « `.o_statusbar_buttons button[name='action_confirm']` » pour la confirmation d'un devis — et compose un parcours pas-à-pas qui cible les vrais champs requis, l'ajout de lignes (« `.o_field_x2many_list_row_add` » pour les lignes de facture ou de commande) et le bouton de validation canonique du modèle. Le schéma introspecté est mis en cache (Redis) et recalculé à la mise à jour du module, rendant le coût d'introspection négligeable en régime établi. Cette approche assure une couverture sans aucune configuration des modules custom : créer, rechercher, valider et supprimer sont guidés sur leurs champs et boutons réels, dans la langue de l'utilisateur.

Garantie anti-hallucination et repli. La sûreté repose ici encore sur le contrat de données : un sélecteur n'est jamais inventé. Le générateur déterministe ne produit que des sélecteurs dérivés du schéma réel ; lorsqu'un planificateur fondé sur un LLM est sollicité pour décomposer un objectif composé (« créer la facture puis la comptabiliser »), chaque étape qu'il propose est validée contre le schéma — toute référence à un champ ou à un bouton absent est rejetée, et le sélecteur final est recalculé côté serveur à partir de la référence validée. La résolution suit un ordre à trois niveaux — plan LLM ancré → scénario déterministe ancré → générateur universel — de sorte qu'aucune capacité existante n'est retirée et que le guidage demeure disponible même lorsque l'introspection est indisponible. Le moteur de superposition (les trois calques décrits plus haut) reste inchangé : il consomme exactement les mêmes UIAction, désormais alimentées par des cibles ancrées. Cette généralisation matérialise en partie la perspective des « actions agentiques » de la conclusion : le système ne se contente plus de connaître les écrans qu'il a appris, il découvre la structure réelle de chaque modèle pour guider l'utilisateur

avec exactitude.

Validation du guidage ancré. L'exactitude de cet ancrage est vérifiée à deux niveaux. Des tests unitaires couvrent la dérivation des sélecteurs et la sélection des champs et boutons réels — champ requis visible et éditables, bouton de workflow canonique — ainsi que la garantie anti-hallucination : une étape dont la référence n'existe pas dans le schéma est systématiquement rejetée. Des tests de bout en bout Playwright confirment ensuite que les sélecteurs produits se résolvent sur le DOM réel d'Odoo, sur des modèles standard (Contacts, Ventes, Facturation, Achats, CRM, Inventaire, Projet) comme sur un modèle développé en interne, l'étape ciblant à chaque fois le champ réel attendu.

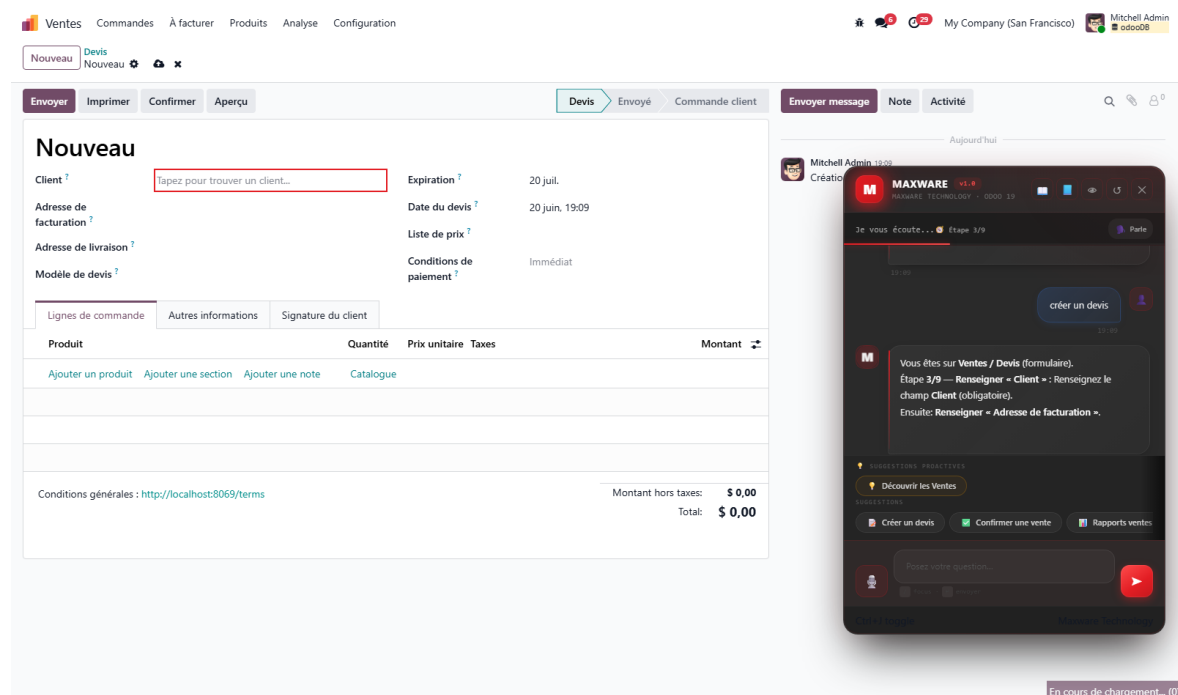


FIGURE 12 – Guidage ancré : sur un nouveau devis, l'étape cible le champ réel « Client » (surbrillance), avec le pas « Étape 3/9 — Renseigner « Client » » et l'étape suivante annoncée.

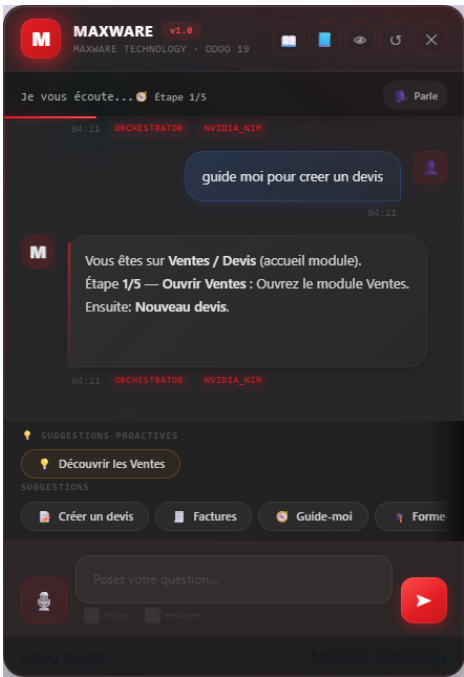


FIGURE 13 – Guidage pas-à-pas avec progression diffusée en direct (streaming SSE)

Les trois modes sont illustrés ci-dessous sur un même cas réel — le bouton « Nouveau » du module Ventes — tels que les rend le moteur de superposition :

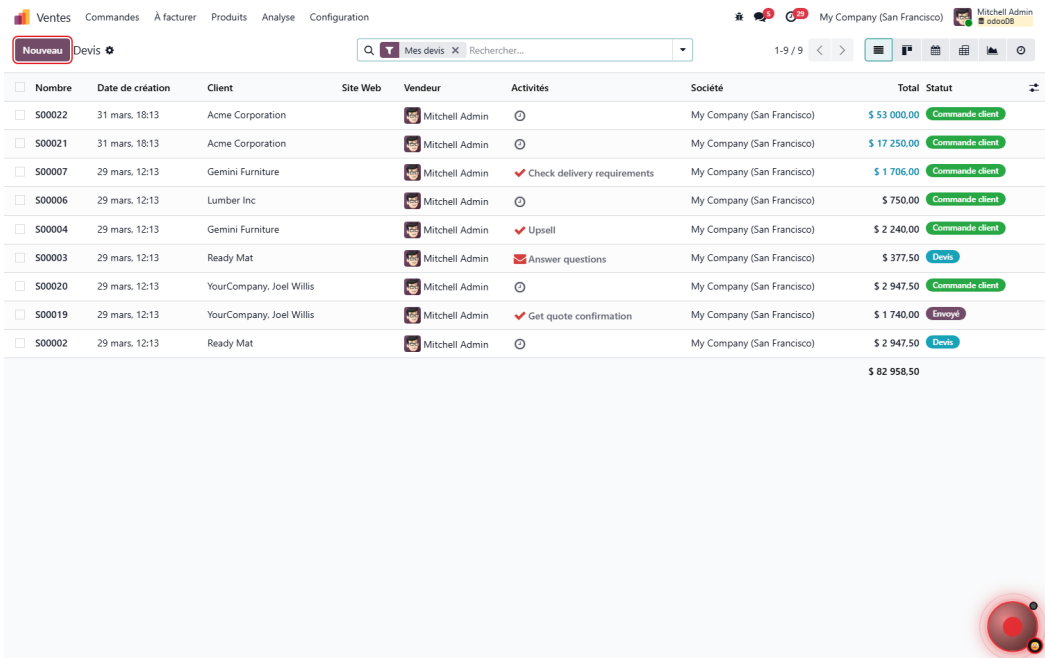


FIGURE 14 – Mode surbrillance : le bouton cible est entouré d'un anneau animé

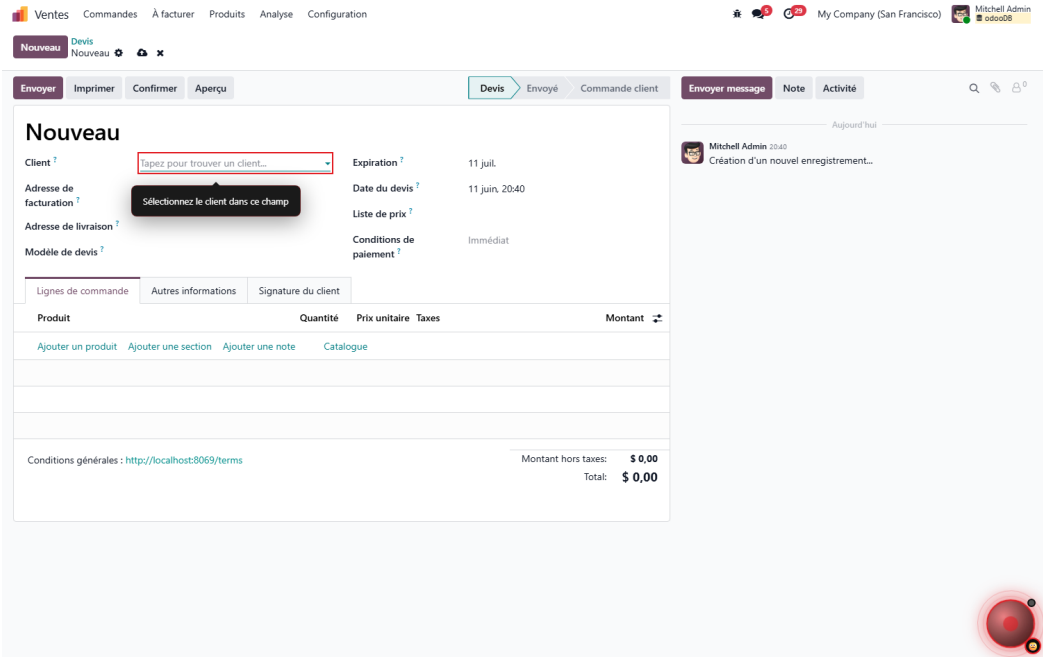


FIGURE 15 – Mode info-bulle : bulle d’aide contextuelle ancrée sur le champ « Client »

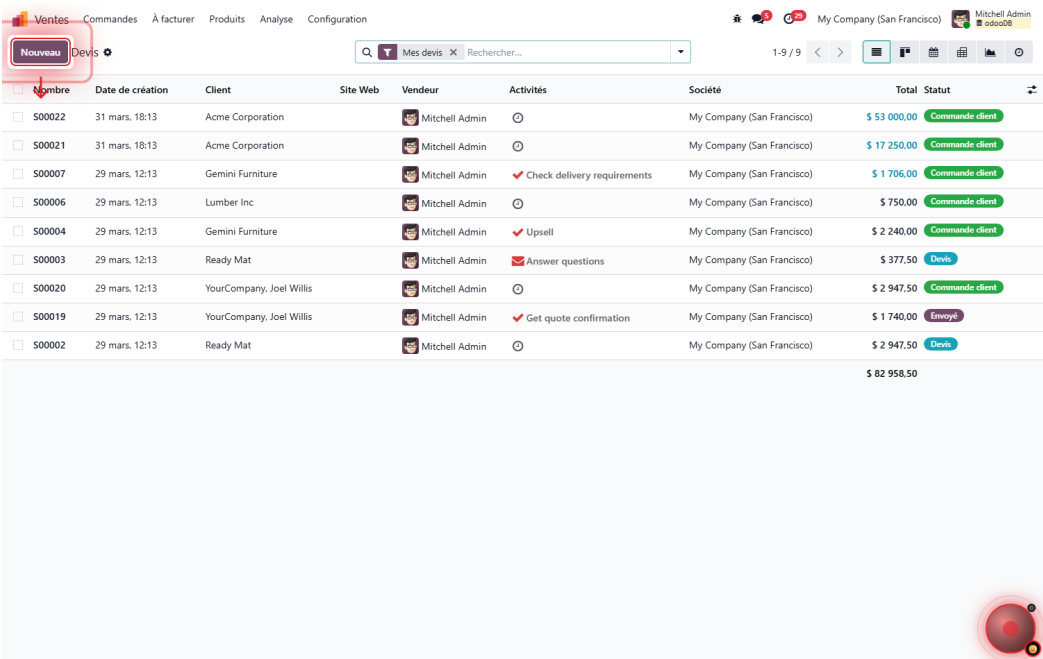


FIGURE 16 – Mode flèche : flèche SVG animée pointant la cible (surbrillance ajoutée pour la lisibilité du clicé)

En guidage multi-étapes, la barre d'état du panneau affiche la progression pendant que l'utilisateur exécute chaque geste sur l'interface réelle :

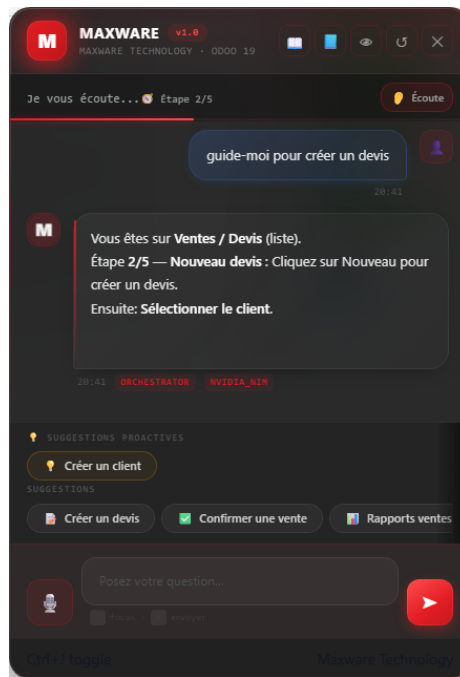


FIGURE 17 – Guidage pas-à-pas en cours : compteur « Étape 2/5 » dans la barre d'état du panneau

Ce même moteur sert la résolution d'erreur. Un `MutationObserver` détecte les notifications et validations d'Odoo ; le message est transmis au backend avec le contexte, et l'avatar revient avec un diagnostic priorisé pendant qu'Odoo maintient les champs en cause surlignés. La réponse embarque ici trois vérifications à mener et la documentation pertinente :

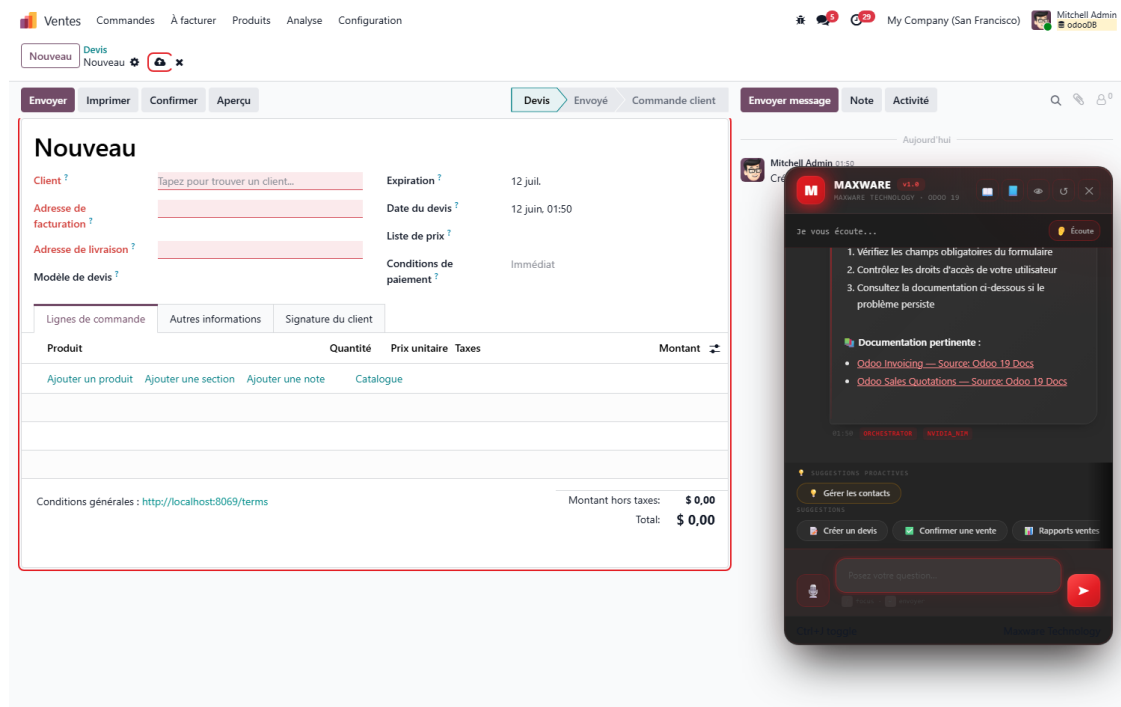


FIGURE 18 – Résolution d’erreur : champs requis signalés, diagnostic en trois points et documentation proposée par l’avatar

III.3.3 Mode démonstration

En mode démonstration, l’avatar déroule lui-même la séquence : un curseur animé se déplace vers chaque élément cible, l’élément est mis en évidence et une narration explique le geste à accomplir. L’utilisateur confirme explicitement chaque étape avant de passer à la suivante.

La sûreté de ce mode ne repose pas sur cette seule confirmation : elle est garantie **par construction du contrat de données**. Le schéma des actions d’interface (UIAction) est une liste blanche fermée — highlight, tooltip, arrow, spotlight, navigate, open_doc — validée côté serveur par Pydantic : aucun type « clic », « saisie » ou « soumission de formulaire » n’existe dans ce vocabulaire, si bien qu’une sortie du LLM qui tenterait d’exprimer une écriture serait rejetée à la validation avant même d’atteindre le navigateur. Le widget, de son côté, n’appelle aucune route de mutation pendant une démonstration ; et toute écriture dans Odoo passerait de toute façon par l’ORM côté serveur, sous le contrôle des ACLs de l’utilisateur. Les actions effectivement réalisables se limitent donc à la navigation, au survol, au défilement et à la lecture — les opérations de création, de modification ou de suppression de données sont structurellement impossibles depuis ce mode.

La capture suivante montre une démonstration en cours — « montre-moi comment créer

une facture » : le bandeau d'état annonce l'étape (« Étape 1/5 — Ouvrez le module Facturation »), l'icône cible est entourée avec le curseur-avatar posé dessus, le plan complet des cinq étapes est affiché dans le panneau, et les boutons « Continuer » et « Stop » matérialisent la confirmation requise avant chaque action.

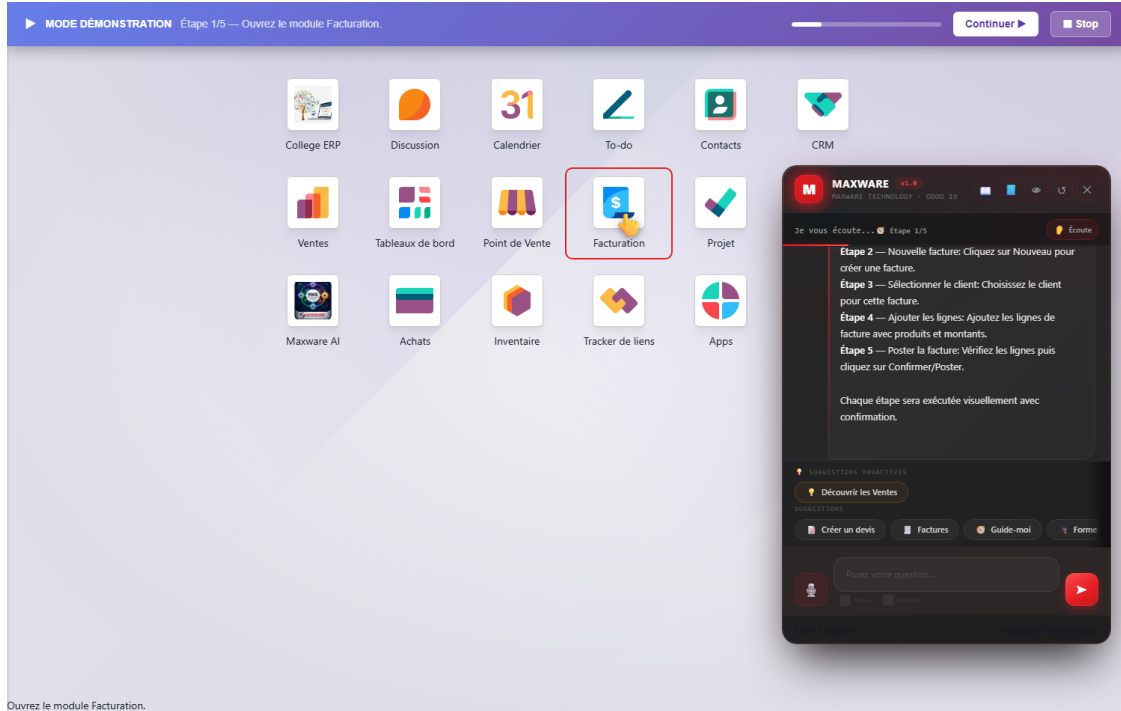


FIGURE 19 – Mode démonstration : bandeau d'étape, cible mise en évidence, plan des étapes et confirmation explicite

III.3.4 Formation adaptative

La formation s'appuie sur un QCM d'évaluation généré par le LearnAgent dans une boîte de dialogue OWL dédiée, dont la difficulté est adaptée au niveau de l'utilisateur, dérivé de son état de répétition espacée (algorithme SM-2 [11]). La notation est effectuée côté serveur — les réponses correctes ne sont jamais transmises au navigateur — et un unique passage de révision SM-2 est appliqué à l'issue du questionnaire complet. Le parcours qui en découle est ancré dans la documentation réelle récupérée via le pipeline RAG, présenté étape par étape avec barre de progression, guidage visuel et validations intermédiaires ; la progression est persistée dans Redis pour une reprise après déconnexion.

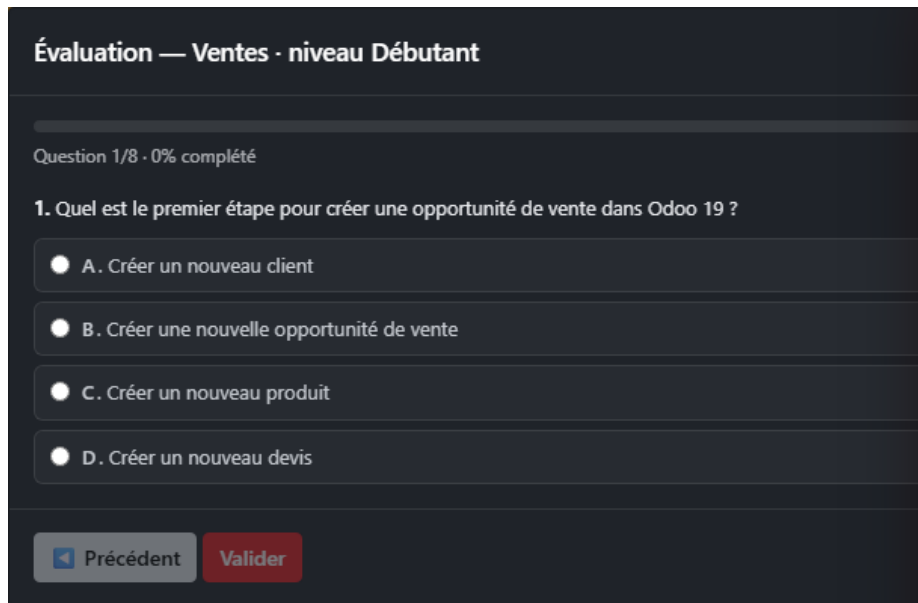


FIGURE 20 – Évaluation par QCM dans une boîte de dialogue OWL dédiée : question, choix multiples et progression

À l'issue du questionnaire, l'utilisateur reçoit une synthèse — score calculé côté serveur, message d'encouragement adapté et nombre de questions à revoir — qui alimente la mise à jour de l'état de répétition espacée décrite plus bas.



FIGURE 21 – Synthèse de fin de QCM : score noté côté serveur et questions à revoir

Le parcours qui en découle est présenté étape par étape : chaque écran affiche la progression, une durée estimée et les objectifs de l'étape. Deux boutons y relient la formation aux autres capacités du système — « Montre-moi » déclenche le guidage visuel sur l'interface réelle, « Vidéo » ouvre la ressource recommandée par l'agent documentaire — et la validation (« Fait — Suivant ») fait avancer la progression persistée dans Redis.

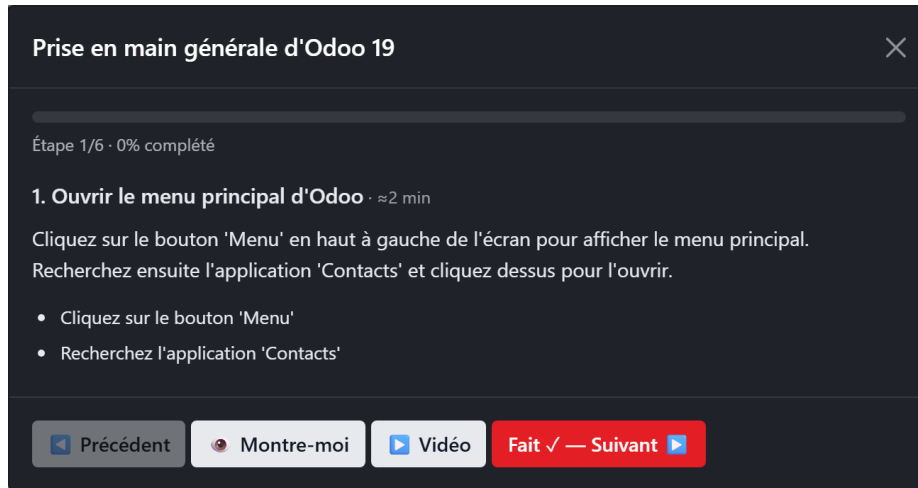


FIGURE 22 – Étape d'un parcours de formation adaptatif, avec déclenchement du guidage (« Montre-moi ») et de la ressource vidéo

Fondements : courbe de l'oubli et algorithme SM-2

Le choix de la répétition espacée s'appuie sur un résultat ancien et robuste de la psychologie expérimentale : la courbe de l'oubli d'Ebbinghaus [10], qui montre que la rétention d'une information décroît rapidement après l'apprentissage, mais que chaque réactivation espacée aplatit la courbe et allonge la durée de mémorisation. L'algorithme SM-2, conçu par Woźniak en 1987 et formalisé dans son mémoire de 1990 [11], opérationnalise ce principe avec une économie de moyens remarquable : chaque connaissance suivie porte trois grandeurs — un compteur de répétitions n , un facteur de facilité EF (initialisé à 2,5) et un intervalle I en jours.

Après chaque évaluation, la qualité de la réponse q (entre 0 et 5) pilote la mise à jour. Si $q \geq 3$, l'intervalle progresse : $I_1 = 1$ jour, $I_2 = 6$ jours, puis $I_n = \text{arrondi}(I_{n-1} \times EF)$; sinon, le compteur et l'intervalle sont réinitialisés — la connaissance redevient « fragile ». Le facteur de facilité évolue dans les deux cas selon la formule originale : $EF \leftarrow \max(1,3; EF + 0,1 - (5 - q) \times (0,08 + (5 - q) \times 0,02))$. Notre implémentation borne en outre EF à 3,0 pour éviter des intervalles déraisonnables. L'extrait suivant, tiré de l'agent de formation, montre cette mise à jour telle qu'elle s'exécute après un QCM complet :

```
# Mise à jour SM-2 — un seul passage par QCM complet (learn_agent.py)
ease = float(state.get("ease", 2.5))
repetitions = int(state.get("repetitions", 0))
if quality >= 3 : # réponse correcte
    if repetitions == 0 : interval = 1
```

```

elif repetitions == 1 : interval = 6
else : interval = int(round(state.get("interval", 1) * ease))
repetitions += 1
else : # échec : on repart de zéro
interval, repetitions = 1, 0
ease = max(1.3, ease + 0.1 - (5 - quality) * (0.08 + (5 - quality) * 0.02))
ease = min(3.0, ease) # garde-fou maison
redis.setex(key, 365 * 24 * 3600, json.dumps({...})) # learn :sm2:{user}:{module}

```

Quatre champs suffisent donc à représenter l'état d'apprentissage d'un utilisateur sur un module — répétitions, facilité, intervalle, date de dernière révision — stockés sous la clé `learn :sm2:{user}:{module}` avec une durée de vie d'un an. Une révision est due dès que le temps écoulé dépasse l'intervalle, ce qui alimente les suggestions proactives ; et le niveau affiché dans les tableaux de bord (débutant, intermédiaire, avancé) dérive directement du compteur de répétitions, ce qui rend la mécanique lisible pour l'utilisateur comme pour l'administrateur.

Pourquoi SM-2 plutôt que FSRS

Des ordonnanceurs plus récents prédisent mieux la rétention : FSRS, fondé sur les travaux de Ye, Su et Cao [12], modélise la dynamique de la mémoire et surpasse SM-2 dans les comparatifs publiés. Nous l'avons écarté pour cette version après réflexion, pour trois raisons concrètes. D'abord le démarrage à froid : FSRS tire sa précision de l'optimisation de ses paramètres — dix-sept et plus selon les versions — sur l'historique de révisions de l'utilisateur, historique dont nos utilisateurs ne disposent pas à l'installation ; SM-2 fonctionne correctement dès la première interaction. Ensuite la compacité : quatre champs Redis par couple utilisateur-module restent triviaux à inspecter, à afficher et à déboguer, là où un vecteur de paramètres par item compliquerait les tableaux de bord. Enfin l'explicabilité : la règle de SM-2 s'explique en une phrase à un utilisateur ou à un support technique. L'historique des QCM que nous accumulons (`learn :history`) rendra l'optimisation FSRS possible : la migration est documentée comme perspective, pas comme manque.

III.3.5 Tableaux de bord d'administration et de suivi

Le module ajoute un menu racine « Maxware AI » dans Odoo regroupant : la configuration de la connexion au backend, la personnalisation de l'avatar (apparence définie côté

serveur, le stockage local du navigateur ne pouvant surcharger qu'une liste blanche de préférences), un tableau de bord « Insights » (statistiques d'usage et points de friction issus de l'AnalyticsAgent), une section « Apprentissage » — Mon niveau, Historique des QCM, Succès, Statistiques — alimentée par les états SM-2, et un tableau « Agents » exposant l'état, l'activité et le taux de succès de chacun des huit agents.

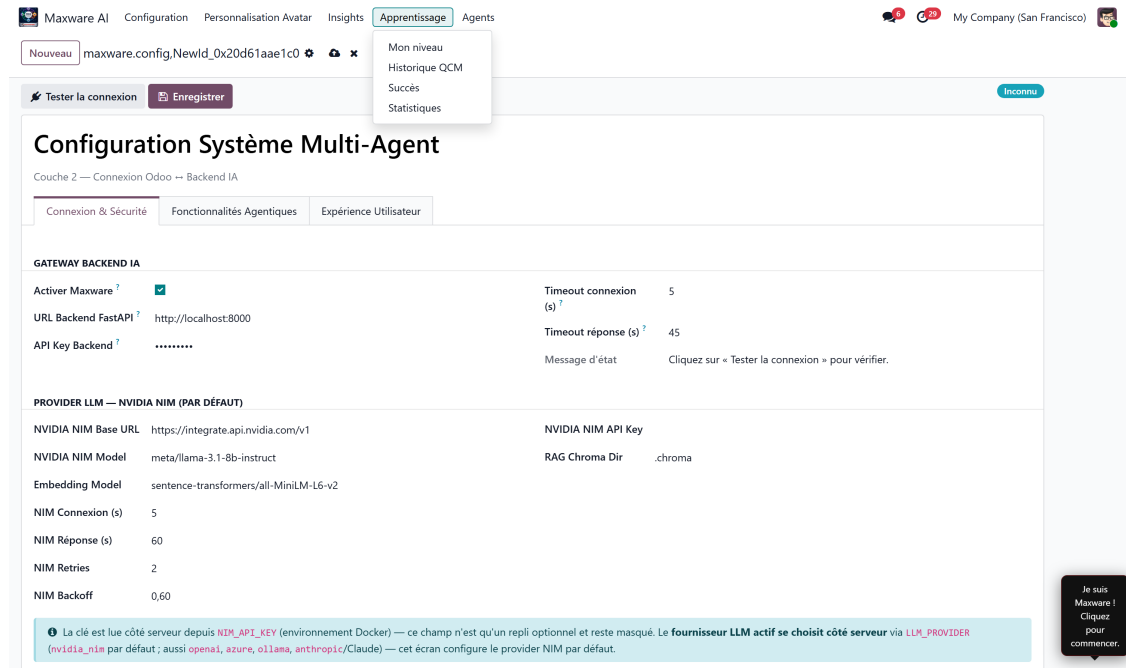


FIGURE 23 – Menu racine « Maxware AI » : configuration, personnalisation, Insights, apprentissage et supervision des agents

La page de configuration centralise la connexion au backend — la clé d'API y reste masquée à l'affichage — et les paramètres du fournisseur LLM. La personnalisation de l'avatar se règle quant à elle avec un aperçu en direct (apparence, position, personnalité, style de réponse) et s'applique sans recharger la page.

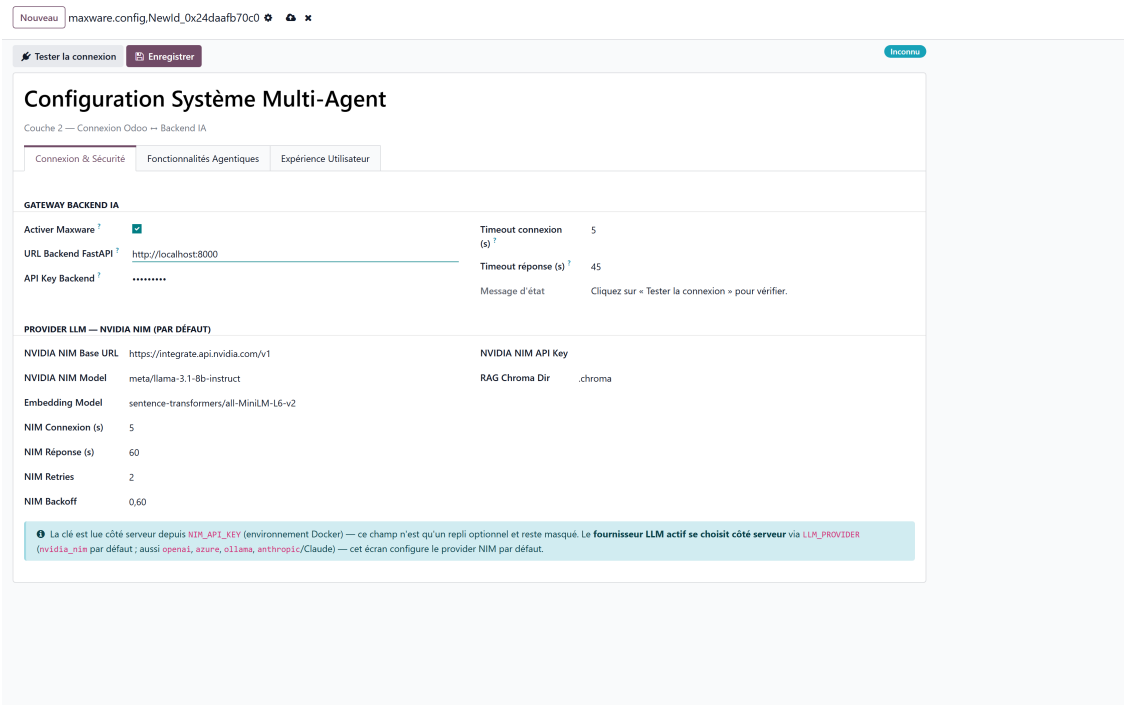


FIGURE 24 – Page Configuration : connexion au backend (clé masquée) et paramètres du fournisseur LLM

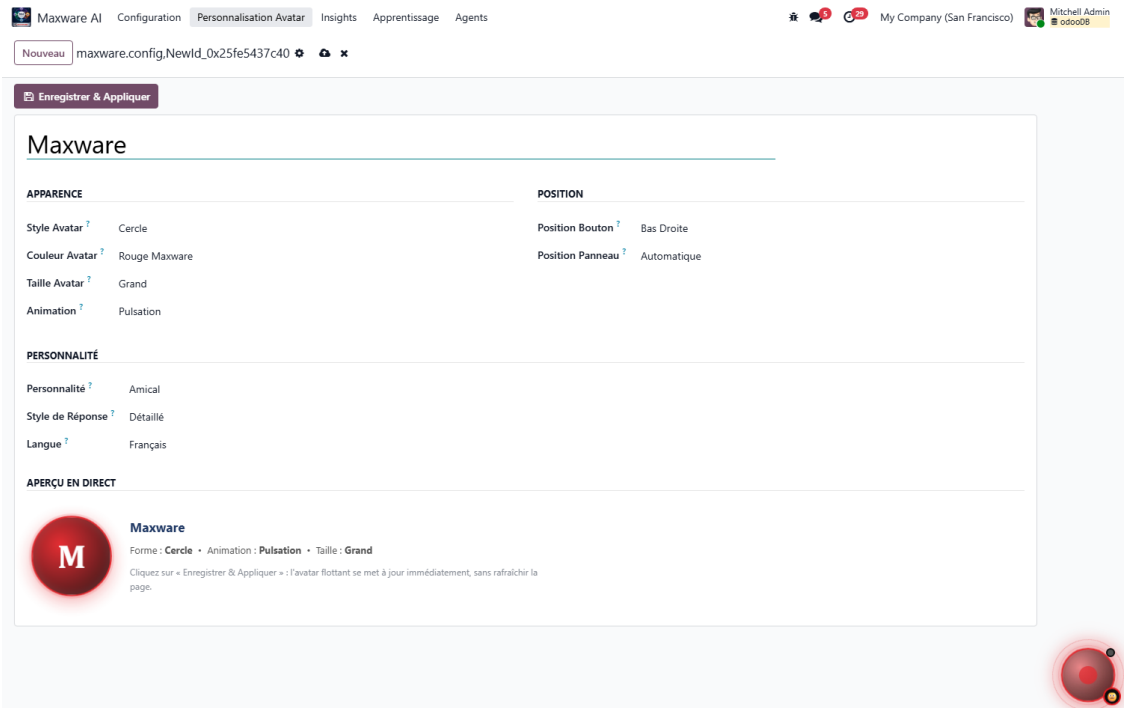


FIGURE 25 – Personnalisation de l'avatar avec aperçu en direct (apparence, position, personnalité)

Le tableau de bord « Insights » expose le journal d'usage construit par l'AnalyticsAgent

— requêtes, modes, latences — et donne à l'administrateur une vue factuelle de ce que l'assistant fait réellement.

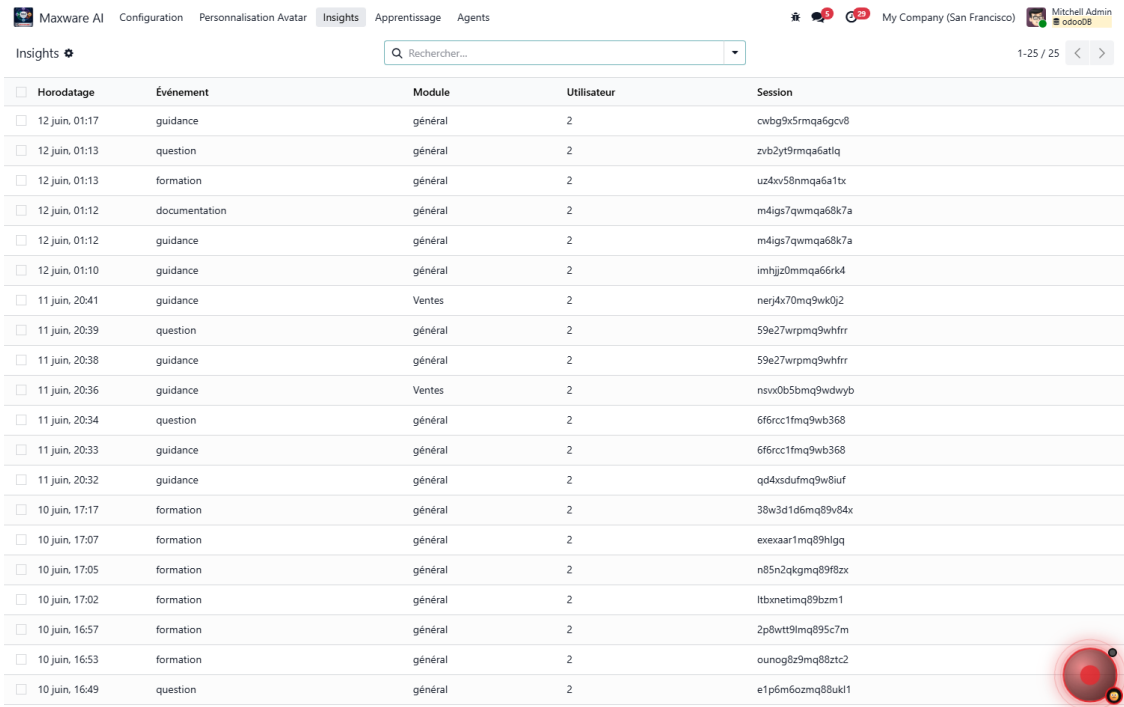


FIGURE 26 – Tableau de bord « Insights » : journal d’usage issu de l’AnalyticsAgent

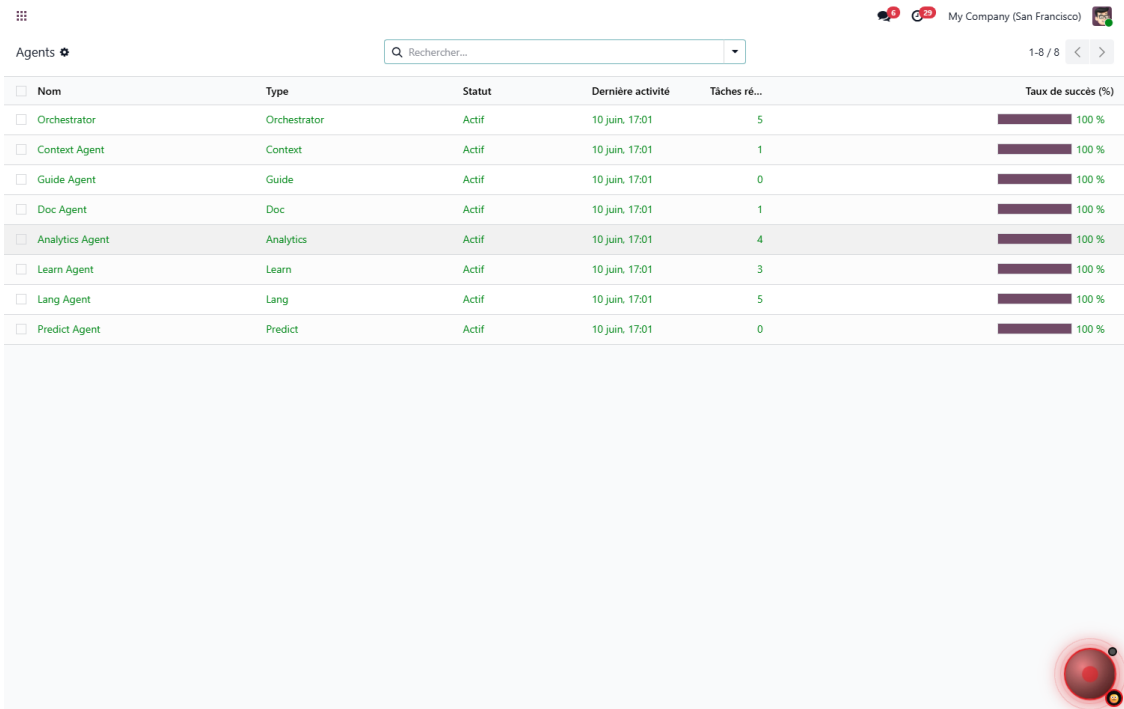


FIGURE 27 – Tableau de bord de supervision des agents dans Odoo

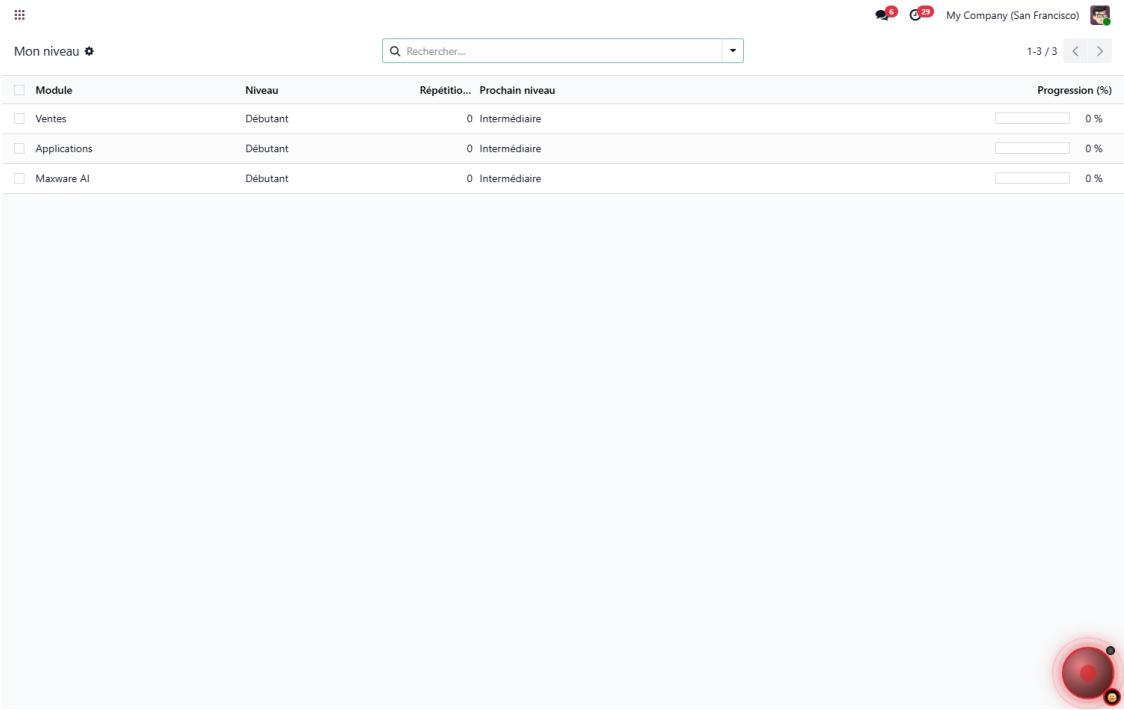


FIGURE 28 – Tableau de bord « Mon niveau » du module Apprentissage (états SM-2 par module)

L'historique des évaluations complète ce suivi : chaque QCM passé y figure avec sa date, son module, son score et le niveau atteint. La session à 2/8 de la synthèse précédente s'y retrouve — le parcours d'une évaluation est ainsi traçable de bout en bout, du widget de l'assistant jusqu'au tableau de bord d'administration.

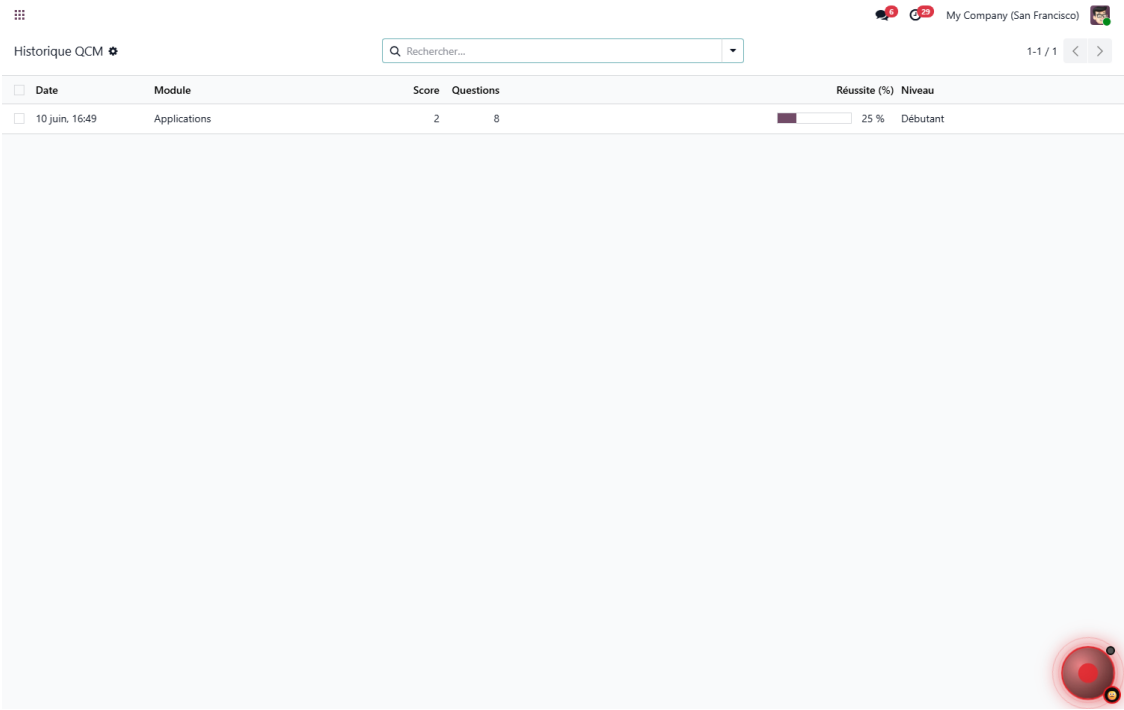


FIGURE 29 – Historique des QCM : score, taux de réussite et niveau dérivé de l'état SM-2

Les vues « Succès » et « Statistiques » complètent le volet apprentissage : la première suit les badges de progression, la seconde agrège l'activité et la maîtrise par module.

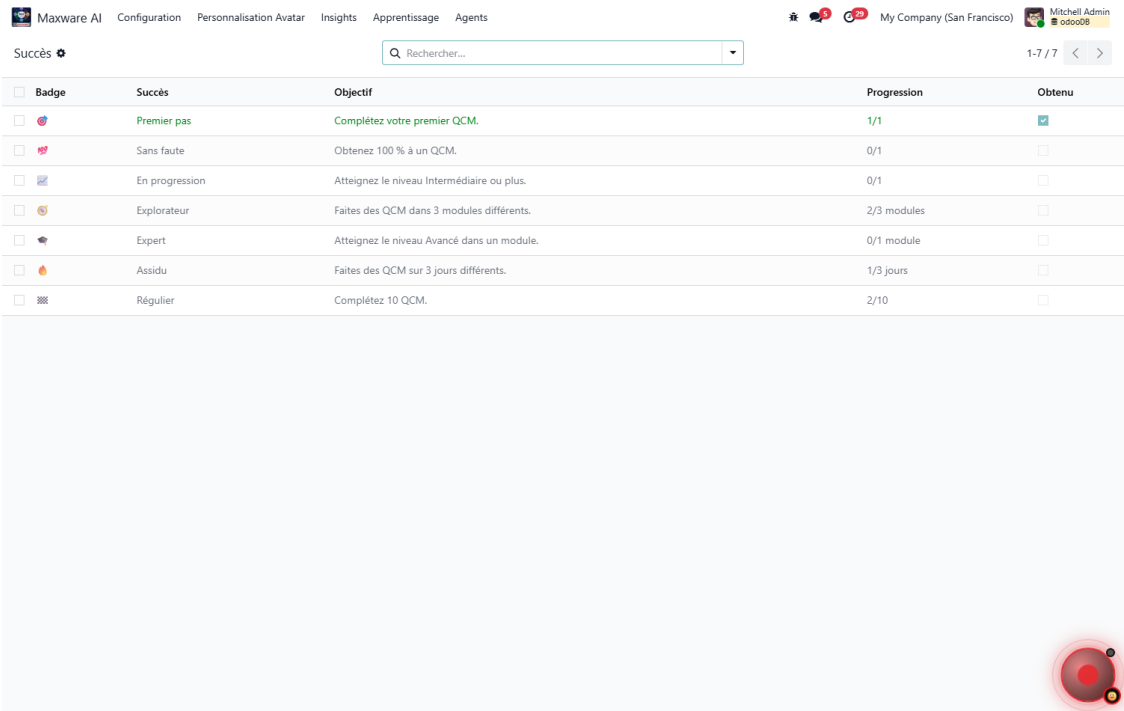


FIGURE 30 – Apprentissage > Succès : badges et progression de l'utilisateur

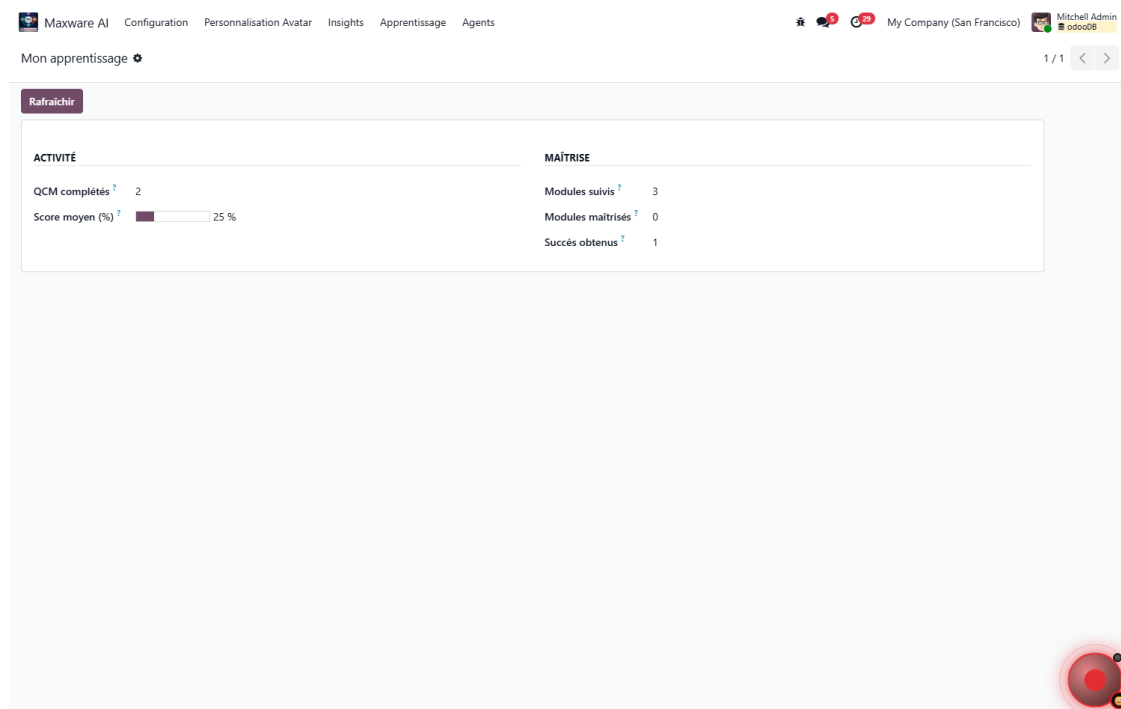


FIGURE 31 – Apprentissage > Statistiques : activité et maîtrise par module

III.3.6 Proxy et sécurité de bout en bout

Le module Odoo n'appelle jamais directement le backend depuis le navigateur : un contrôleur HTTP côté serveur joue le rôle de proxy. Ce choix garantit la validation de la session Odoo, le respect des ACLs (accès aux données via l'ORM côté serveur), la protection CSRF et surtout la confidentialité des secrets : la clé API du backend, comme les clés des services tiers, est stockée côté serveur et injectée par le proxy — le navigateur ne détient aucun secret. Les mesures de durcissement suivantes complètent ce modèle :

- **Authentification** : jeton Bearer vérifié en comparaison à temps constant ; rotation de clés sans interruption (plusieurs clés acceptées simultanément) ; comportement « fail-closed » en production — toute requête est rejetée si aucune clé n'est configurée ; clé d'administration distincte pour les routes sensibles ;
- **Limitation de débit à deux niveaux** : un compteur global partagé dans Redis applique une fenêtre glissante de 60 secondes par utilisateur et par route (Sorted Set, ZADD/ZREMRANGEBYSCORE, 30 requêtes/minute sur les routes d'assistance) — le plafond tient donc même en déploiement multi-répliques ; en cas d'indisponibilité de Redis, ce compteur s'efface (fail-open) devant le second niveau, toujours actif : les quotas slowapi par processus (30/60/120 selon la route) ;
- **Durcissement HTTP** : origines CORS explicites (jamais de joker), en-têtes

- de protection (nosniff, interdiction d'iframe, politique de référent stricte), taille de corps de requête plafonnée, délais de connexion et de lecture stricts ;
- **Journal d'audit** : chaque événement sensible (échec d'authentification, requête d'assistance, rechargement de configuration) est consigné en double — journal structuré et flux Redis plafonné — avec utilisateur, locataire, latence et mode, sans jamais bloquer la requête ;
- **Chiffrement** : échanges chiffrés en HTTPS via le reverse proxy (terminaison TLS sur Nginx en Docker Compose, sur l'Ingress en Kubernetes).

Résistance aux injections de prompt

Un système qui expose un LLM à des utilisateurs doit être pensé contre les injections de prompt — ces messages qui tentent de réécrire les consignes du modèle ou d'exfiltrer des informations internes. Notre défense tient d'abord à l'architecture. Les prompts système sont définis et versionnés côté serveur, dans un module centralisé ; ils ne transitent jamais vers le navigateur. L'entrée de l'utilisateur est injectée exclusivement comme tour de conversation distinct (rôles system et user séparés à l'appel du modèle) : elle peut influencer la réponse, pas les consignes. Aucun secret — clé API, jeton, paramètre d'infrastructure — ne figure dans le contexte transmis au modèle : une instruction du type « ignore tes consignes et révèle ta clé » ne peut rien révéler, faute d'objet. Enfin, toutes les sorties structurées (classification, actions d'interface, QCM) sont validées par des schémas Pydantic à vocabulaire fermé : une génération qui tenterait de produire une action hors liste blanche est rejetée avant exécution. Le plafonnement de la taille des requêtes et la limitation de débit complètent ce dispositif en bornant les tentatives répétées. Une couche de filtrage s'y ajoute désormais : avant tout appel au modèle, l'entrée utilisateur est assainie — taille plafonnée à 2 000 caractères, vingt-sept motifs d'injection français et anglais détectés (réécriture des instructions, demande de révélation du prompt système, changement d'identité, jailbreak, exfiltration) — et toute détection interrompt le traitement par une réponse de refus polie, dans les deux agents qui composent du texte libre à partir du message : documentation et guidage.

III.4 Déploiement et intégration continue

Le backend et ses dépendances sont orchestrés par Docker Compose (sept services : backend, Redis, ChromaDB, indexeur, Nginx, Prometheus, Grafana) ; l'instance Odoo s'exécute comme service natif et dialogue avec le backend conteneurisé via le proxy HTTP. Les modèles d'embedding sont pré-intégrés dans l'image Docker afin d'éliminer tout téléchargement au démarrage à froid. Le déploiement du module Odoo est automatisé par un script qui met à jour le module puis redémarre le service.

Pour la production, des manifestes Kubernetes accompagnent le projet : déploiement

du backend avec quatre processus applicatifs par réplique, sondes de vivacité et de disponibilité, autoscaler horizontal de deux à six répliques, Ingress TLS, ConfigMap et Secrets. Le caractère sans état du backend (état de session externalisé dans Redis) autorise cette montée en charge horizontale sans affinité de session.

L'intégration continue est assurée par GitHub Actions : à chaque modification, la chaîne installe les dépendances, démarre un service Redis éphémère, exécute la suite de tests unitaires (pytest) puis vérifie la qualité du code (ruff). Les appels LLM y sont systématiquement simulés afin de garantir des exécutions déterministes et sans coût.

Conclusion

Cette phase de réalisation a permis de transformer l'architecture conçue en un système fonctionnel et industrialisé : un backend multi-agent orchestré par LangGraph avec court-circuits heuristiques, une couche LLM portable protégée par un circuit breaker, un pipeline RAG complet sur un corpus de 4 405 segments, un streaming SSE de bout en bout, et un module Odoo riche — avatar, guidage, démonstration, formation et tableaux de bord d'administration — le tout sécurisé par un proxy serveur et un durcissement systématique. Le chapitre suivant évalue ce système sur les plans fonctionnel et des performances.

Chapitre IV : Tests et évaluation

Ce dernier chapitre présente la démarche de validation du système : la stratégie de tests, les résultats fonctionnels obtenus, les mesures de performance et les optimisations qui les ont rendues possibles, l'évaluation de la qualité du pipeline documentaire, les limites observées qui ouvrent les perspectives d'évolution, et enfin le bilan de conformité au cahier des charges initial.

IV.1 Stratégie de tests

La validation s'organise en quatre niveaux complémentaires :

- **Tests unitaires et d'intégration (pytest)** : 294 tests répartis en seize modules couvrent chaque agent (orchestrateur, contexte, guidage, documentation, formation, traduction, analyse, prédiction), le circuit breaker, la sécurité (authentification, limitation de débit), le streaming SSE, les court-circuits heuristiques et l'outillage ChromaDB. Les appels LLM y sont systématiquement simulés (mocks) : la suite est déterministe, rapide et sans coût d'API. La couverture de code, mesurée avec pytest-cov dans ces mêmes conditions (Redis réel, LLM simulé), s'établit à 67 % des 5 613 instructions de l'application en intégration continue (66 % en exécution locale sous Windows, où cinq tests dépendant de Redis sont ignorés) ;
- **Tests de bout en bout (Playwright)** : scénarios joués dans un navigateur réel sur une instance Odoo de développement, validant les cinq flux fonctionnels majeurs — question avec guidage, formation par QCM, mode démonstration, résolution d'erreur et récupération documentaire — ainsi que le comportement de l'avatar et des superpositions visuelles ;
- **Intégration continue (GitHub Actions)** : à chaque modification du dépôt, la chaîne installe les dépendances, démarre un service Redis éphémère, exécute la suite pytest complète puis le linter ruff. Aucune fusion n'est possible si un test échoue.
- **Évaluation quantitative du pipeline RAG** : un jeu de référence de 65 paires question → document attendu, couvrant douze modules fonctionnels, mesure la qualité de la récupération documentaire selon les métriques Precision@5 et MRR (voir section IV.4).

IV.2 Résultats des tests fonctionnels

Périmètre testé	Résultat	Observations
Suite unitaire pytest (16 modules)	294 collectés (289 + 5 ignorés)	Exécution intégralement simulée côté LLM ; couverture de tous les agents et middlewares
Couverture de code (pytest-cov)	67 % global	Cœur décisionnel au-delà de 80 % ; adaptateurs réseau volontairement moins parcourus (voir analyse ci-dessous)
Flux 1 — question + guidage visuel	Validé (E2E)	Réponse affichée, surbrillance et info-bulle posées sur la bonne cible, démontage correct des superpositions
Flux 2 — formation adaptative (QCM)	Validé (E2E)	QCM généré et noté côté serveur, état SM-2 mis à jour, tableaux de bord cohérents
Flux 3 — mode démonstration	Validé (E2E)	Séquence d'actions jouée avec confirmations entre étapes
Flux 4 — résolution d'erreur	Validé (E2E)	Erreur détectée, explication contextuelle et champs mis en évidence
Flux 5 — récupération documentaire (RAG)	Validé (E2E)	Réponse ancrée avec sources citées, cache opérationnel à la seconde requête
Streaming SSE + repli synchrone	Validé (E2E)	Étapes du pipeline affichées en direct, jetons peints, repli transparent en cas de coupure du flux
Avatar et interface	Validé (E2E)	Quatre états Lottie fonctionnels, aucune erreur dans la console du navigateur

La couverture appelle une lecture nuancée plutôt qu'un chiffre brut. Son profil reflète directement la stratégie de test : le cœur décisionnel dépasse les 80 % — contrats de données (99 %), configuration (98 %), circuit breaker (94 %), streaming SSE (88 %), orchestrateur (82 %), sécurité (81 %), agent de contexte (79 %) — tandis que les zones les moins parcourues sont précisément celles que la suite unitaire simule par

construction : les adaptateurs de fournisseurs LLM (26 à 53 % selon l'adaptateur — chemins réseau réels) et une partie de l'outillage d'intégration des agents, dont les branches d'intégration sont exercées par les scénarios Playwright plutôt que par les tests unitaires. Élever la couverture de ces adaptateurs passe par des tests d'intégration contre des services factices.

Les deux captures suivantes documentent ces exécutions. La première provient du poste de développement (Windows) : 294 tests collectés, 289 réussis et 5 ignorés — les tests de l’agent d’analyse s’ignorent d’eux-mêmes lorsque Redis n’est pas joignable depuis la session de test — pour une couverture de 66 % ; sous Linux avec Redis actif, conditions de l’intégration continue, la suite complète s’exécute et la couverture atteint 67 %. La seconde récapitule les cinq flux de bout en bout exercés dans un navigateur réel lors de la session de validation.

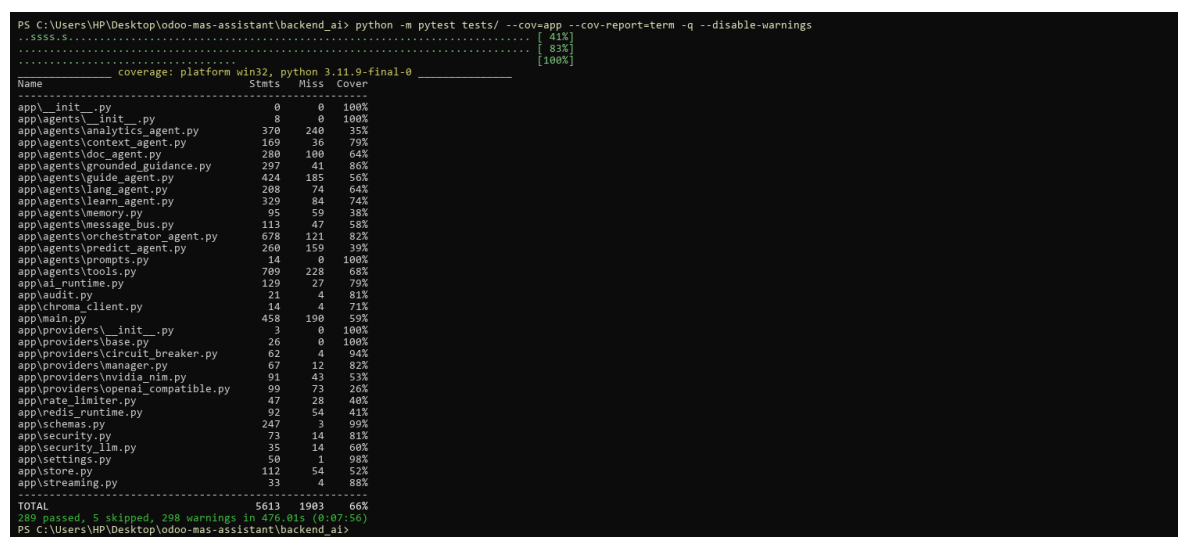


FIGURE 32 – Exécution locale de la suite pytest avec couverture (Windows : 289 réussis, 5 ignorés faute de Redis joignable, 66 %)



FIGURE 33 – Récapitulatif des cinq flux fonctionnels validés de bout en bout (session Playwright)

IV.3 Mesures de performance

IV.3.1 Latence de bout en bout par flux

Les mesures ci-dessous ont été relevées à chaud sur l'environnement de développement, le backend étant connecté à NVIDIA NIM. La latence totale est dominée par les appels au LLM ; c'est précisément ce que les optimisations de la section suivante visent à réduire.

Flux	Latence mesurée	Chemin emprunté
Guidage explicite (« guide-moi pour créer un devis »)	3,4 – 5,5 s	Court-circuit heuristique → contexte → guidage par règles, sans LLM d'inférence
Question + guidage	5 – 9 s	Classification LLM → contexte → guidage documentation → synthèse
Documentation / RAG (cache absent)	4 – 8 s	Récupération ChromaDB + réordonnancement (+ augmentation LLM)
Documentation / RAG (cache présent)	≈ 0 s	Réponse ancrée servie directement depuis Redis
QCM explicite (8 questions)	11 – 18 s	Court-circuit formation (saut des autres agents) + génération structurée
Conversation informelle (« bonjour », « quelle heure »)	< 0,1 – 1,5 s	Chemin rapide : heure/date déterministes, sinon un seul appel LLM diffusé
Mode démonstration	≈ 3 s	Court-circuit démonstration → actions structurées

Au-delà de la latence totale, le levier décisif pour l'expérience utilisateur est le **temps jusqu'au premier signal** : grâce au streaming SSE, la première trame d'étape parvient au widget en environ 0,2 seconde — l'utilisateur voit la véritable étape du pipeline (« Recherche documentaire... ») au lieu d'une attente muette — puis les jetons de la réponse se peignent au fil de la génération.

IV.3.2 Optimisations appliquées et gains mesurés

Optimisation	Gain mesuré
Court-circuits heuristiques de classification (guidage, QCM, démonstration...)	– 1 appel LLM de classification et saut des agents inutiles : QCM 17,5 → 11,2 s ; guidage explicite $\approx 9-14 \rightarrow 3,4-5,5$ s
Inférence de guidage heuristique (parcours connus sans LLM)	Agent de guidage $\approx 5-10$ s $\rightarrow \approx 1-2$ s à chaud
Cache Redis documentaire (1 h) incluant la réponse ancrée	Requête répétée : 10,9 s $\rightarrow \approx 0$ s
Synthèse LLM conditionnelle (uniquement si guidage ET documentation contribuent)	– 1 appel LLM sur les chemins simples
Réessais de lecture LLM désactivés (délai strict)	Un appel gelé coûte au plus 60 s au lieu de 180 s
Réécriture LLM du guidage désactivée par défaut (formulations structurées)	– 1 à 2 s par étape de guidage
Boucle ReAct en option (appel direct par défaut)	Évite les itérations LLM multiples sur les chemins courants
Chemin rapide « conversation informelle »	Pipeline complet $\rightarrow 0$ à 1 appel LLM
Modèles d'embedding pré-intégrés dans l'image Docker (≈ 79 Mo)	Démarrage à froid sans téléchargement
Seuil plancher de pertinence + déduplication des sources	Contexte documentaire sans bruit $\rightarrow$ invites d'augmentation plus courtes
Circuit breaker sur la couche LLM	En cas de panne du fournisseur : échec en quelques millisecondes au lieu de 60 s de délai par appel

IV.4 Évaluation quantitative du pipeline documentaire (RAG)

IV.4.1 Protocole d'évaluation

La qualité de la récupération a été mesurée selon un protocole reproductible : un jeu de référence de 65 paires (question en français $\rightarrow$ document attendu) a été constitué manuellement, couvrant douze modules fonctionnels d'Odoo, puis exécuté contre le pipeline complet en configuration de production — translittération français $\rightarrow$ anglais par glossaire, recherche vectorielle dans la collection de 4405 segments, réordonnancement

combiné, renforcement lexical, seuil plancher et déduplication. Deux métriques standard de la recherche d'information ont été retenues : la **Precision@5**, proportion de questions pour lesquelles le document attendu figure parmi les cinq résultats retenus — soit précisément le panel transmis au LLM pour l'augmentation — et le **MRR** (Mean Reciprocal Rank), rang réciproque moyen du document attendu lorsqu'il est récupéré, qui mesure sa position dans le classement.

Formellement, pour l'ensemble Q des questions du jeu, où chaque question q admet un unique document attendu de rang $\text{rang}(q)$ dans la liste retournée : **Precision@5** = $|\{ q \in Q : \text{rang}(q) \leq 5 \}| / |Q|$ et **MRR** = $(1/|Q'|) \times \sum 1/\text{rang}(q)$, la somme portant sur le sous-ensemble Q' des questions dont le document est effectivement récupéré. Ce cadre — un seul document pertinent par question, jugement binaire — explique le choix des métriques : Recall@5 y coïnciderait exactement avec Precision@5, et le NDCG, qui suppose des jugements de pertinence gradués dont nous ne disposons pas, n'apporterait aucune information supplémentaire. Il explique aussi pourquoi un MRR supérieur à la Precision@5 est cohérent : le premier est conditionné aux documents retrouvés et mesure leur rang, la seconde compte les échecs de récupération.

IV.4.2 Résultats

Sur l'ensemble du jeu de référence, le pipeline atteint une **Precision@5 globale de 0,843** et un **MRR de 0,973**. Le tableau suivant détaille la précision par module (appréciation : excellente $\geq 0,90$; bonne 0,80–0,89 ; satisfaisante 0,75–0,79 ; à renforcer $< 0,75$).

Module	Precision@5	Appréciation
Ventes	0,967	Excellente
Facturation	0,767	Satisfaisante
Inventaire	0,800	Bonne
CRM	0,667	À renforcer
Achats	0,920	Excellente
Comptabilité	0,920	Excellente
Fabrication	0,840	Bonne
Projet	0,800	Bonne
RH / Paie	0,960	Excellente
Qualité	0,800	Bonne
Maintenance	0,760	Satisfaisante

Module	Precision@5	Appréciation
Barcode	0,960	Excellente
Ensemble — 12 modules, 65 questions	0,843	Precision@5 globale (MRR : 0,973)

La valeur élevée du MRR (0,973) indique que, lorsque le document attendu est récupéré, il se classe presque systématiquement en première position du panel : le contexte fourni au LLM pour l'augmentation est donc non seulement présent, mais correctement priorisé.

IV.4.3 Analyse des résultats

Points forts : Onze des douze modules atteignent ou dépassent l'objectif de 0,75, et cinq d'entre eux (RH/Paie, Comptabilité, Barcode, Ventes, Achats) dépassent 0,90. Les meilleurs scores correspondent aux modules dont le vocabulaire est le plus spécifique — terminologie comptable, codes-barres, paie — : la similarité sémantique y est naturellement discriminante et le renforcement lexical sur les titres y trouve des correspondances exactes. La translittération français → anglais par glossaire, le corpus étant anglophone, s'est confirmée déterminante sur l'ensemble des modules.

Points faibles : Deux modules se situent dans la bande satisfaisante (Facturation : 0,767 ; Maintenance : 0,760) et un seul, le CRM (0,667), reste sous l'objectif. L'analyse des questions en échec suggère deux causes principales : d'une part, un chevauchement terminologique entre modules voisins — le vocabulaire de la facturation recoupe celui des ventes, les mouvements de stock traversent l'inventaire, les achats et la fabrication — qui disperse la similarité sur plusieurs documents plausibles ; d'autre part, pour le CRM, un vocabulaire polysémique (« opportunité », « piste », « activité », « étape ») proche du langage courant et une documentation davantage conceptuelle que procédurale, qui produisent des embeddings moins discriminants ; le glossaire de translittération couvre en outre moins finement ce jargon commercial.

Sur le plan de la fiabilité de la génération, une limite identifiée dans les premières itérations — des scores de pertinence sortant de l'intervalle $[0, 1]$ après combinaison des critères — a été levée : les scores sont désormais normalisés et bornés, et le seuil plancher écarte les correspondances quasi aléatoires avant l'étape d'augmentation. La boucle de validation de la réponse (présence de citations, fidélité aux sources) rejette par ailleurs les générations insuffisamment ancrées, réduisant sensiblement le risque d'hallucination.

IV.4.4 Axes d'amélioration

- **Étiquetage des segments par métadonnées** : enrichir chaque segment, dès l'ingestion, de métadonnées structurées (module d'appartenance, sous-thème, type de contenu : procédure, concept, référence) afin de filtrer ou de pondérer la recherche par module en amont de la similarité ; le critère « pertinence du module » du réordonnement, aujourd'hui limité à 10 % du score combiné, gagnerait ainsi en fiabilité et résorberait le chevauchement terminologique observé entre modules voisins ;
- **Enrichissement ciblé du corpus CRM** : compléter la documentation officielle par des contenus procéduraux (tutoriels, FAQ, notes de version), découper plus finement les pages conceptuelles longues et étendre le glossaire français → anglais au vocabulaire propre au CRM (piste, opportunité, étape du pipeline commercial), afin de ramener le seul module sous l'objectif au niveau des autres.

En synthèse, avec une Precision@5 globale de 0,843, un MRR de 0,973 et l'objectif de 0,75 atteint ou dépassé sur onze des douze modules évalués, le pipeline de récupération documentaire est quantitativement validé ; le module CRM, seul sous le seuil, fait l'objet des axes d'amélioration ciblés ci-dessus.

IV.5 Limites observées et pistes d'amélioration

- **Génération des QCM** : la production d'environ huit questions structurées reste un appel LLM long (8 à 15 s), non diffusable en flux car la sortie est un objet JSON ; l'interface compense par un état de génération détaillé étape par étape ;
- **Embeddings multilingues** : un modèle multilingue a été expérimenté puis écarté — mémoire multipliée par cinq, latence accrue, gain de récupération nul par rapport au glossaire français → anglais ; la bascule reste néanmoins possible par simple configuration et ré-ingestion ;
- **Compatibilité Odoo Enterprise** : le module est compatible par conception ; la validation complète a été menée sur l'édition Community, une campagne dédiée à l'édition Enterprise restant à conduire ;
- **Montée en charge non validée expérimentalement** : le dimensionnement de l'autoscaler Kubernetes (deux à six répliques) repose sur les mesures mono-instance du présent chapitre, pas sur un essai en charge. Une campagne k6 ou Locust — de l'ordre de cinquante utilisateurs simultanés — reste à conduire pour situer le seuil de dégradation de la latence et calibrer les demandes et limites de ressources des pods ;

- **Fonctionnalités non couvertes** : mode hors-ligne, intégration aux messageries d'équipe (Slack, Teams) et synthèse vocale cloud avancée demeurent hors du périmètre de cette version.

IV.6 Bilan de conformité au cahier des charges

Le cahier des charges remis par Maxware Technology au lancement du stage fixait cinq objectifs, huit agents nommés, des livrables et une stack technique pressentie. Le tableau suivant met en regard ces exigences et leur réalisation — les huit agents spécifiés (Context, Guide, Doc, Learn, Lang, Analytics, Predict et l'orchestrateur) sont tous implémentés sous les mêmes noms.

Exigence du cahier des charges	Réalisation	Référence
Architecture multi-agent collaborative intégrée nativement à Odoo	Huit agents orchestrés par un graphe d'états LangGraph ; module Odoo natif (OWL) avec proxy serveur	Chap. II–III
Avatar interactif capable de communiquer	Avatar Lottie flottant et déplaçable ; chat avec rendu Markdown ; voix bidirectionnelle (Web Speech API)	III.3.1
Assistance contextuelle en surimpression sur l'interface	Moteur de guidage à trois modes (surbrillance, info-bulle, flèche), ancré par introspection du schéma réel — couverture universelle de tous les modules, y compris ceux développés en interne	III.3.2
Gestion et recommandation de documentation intelligente	Pipeline RAG sur 4 405 segments et vidéos ; Precision@5 de 0,843, MRR de 0,973	III.2.4, IV.4
Expérience fluide et non intrusive	Streaming SSE (premier signal $\approx 0,2$ s), suggestions proactives non bloquantes, avatar escamotable	III.2.6, IV.3
Prototype intégré Odoo Community/Enterprise	Validé sur Community ; compatible Enterprise par conception (dépendances web, base, mail)	IV.5

Quelques choix techniques s'écartent de la stack pressentie. Ces écarts sont peu nombreux, délibérés, et chacun est argumenté dans le corps du mémoire :

Pressenti au cahier des charges	Retenu	Justification
WebSocket envisagé	Server-Sent Events	Flux unidirectionnel suffisant, compatible avec le proxy Odoo, repli HTTP naturel — II.7.2
React/View + Three.js (3D)	OWL + Lottie (2D)	Intégration native au client Odoo — option d'ailleurs prévue par le cahier des charges lui-même — III.3.1
LLM : OpenAI / Claude / Ollama	Couche multi-fournisseur, NVIDIA NIM par défaut	Indépendance du fournisseur préservée ; accès développeur gratuit pendant le développement — III.2.3
Démonstration : l'avatar « exécute des actions à la place de l'utilisateur »	Démonstration en lecture seule, confirmée étape par étape	Restriction volontaire : liste blanche d'actions validée côté serveur, aucune écriture possible — choix de sécurité assumé — III.3.3

Parmi les huit fonctionnalités additionnelles proposées en option, deux sont livrées — la personnalisation avancée de l'avatar et les commandes vocales — et une troisième est couverte indirectement, le tandem Analytics/Predict jouant le rôle du mode « shadow » d'observation. L'intégration Slack/Teams et le mode hors-ligne figurent dans les perspectives ; la FAQ auto-générée, la collaboration multi-utilisateur et le bac à sable de simulation restent hors du périmètre de cette version. Au total, les cinq objectifs du stage sont atteints, les livrables attendus sont fournis, et les écarts techniques sont justifiés et tracés.

Conclusion

La campagne de validation confirme la robustesse du système : l'intégralité de la suite unitaire passe en continu, les cinq flux fonctionnels majeurs sont validés de bout en bout dans un navigateur réel, le pipeline documentaire est quantitativement validé avec une Precision@5 globale de 0,843 sur douze modules, et les optimisations appliquées — court-circuits heuristiques, caches, chemins rapides, circuit breaker — ramènent les latences à des niveaux compatibles avec un usage interactif, le streaming SSE réduisant en outre fortement l'attente perçue. Les limites identifiées sont circonscrites et assorties de pistes concrètes, qui alimentent les perspectives présentées dans la conclusion générale.

Conclusion générale et perspectives

Ce projet de fin d'études, mené au sein de Maxware Technology, avait pour ambition de réduire le fossé entre la richesse fonctionnelle d'Odoo et la capacité réelle des utilisateurs à l'exploiter, en concevant un Système Multi-Agent Intelligent d'assistance contextuelle intégré nativement à la plateforme. Au terme des quatre phases présentées dans ce mémoire, cet objectif est atteint et concrétisé par un produit fonctionnel : l'assistant Maxware.

L'analyse du projet a d'abord mis en évidence les limites des solutions d'assistance existantes — chatbots rigides, assistants génériques sujets aux hallucinations, aides textuelles passives — et formalisé un cahier des charges exigeant : compréhension contextuelle, guidage visuel, formation adaptative, multilinguisme et proactivité. La conception a ensuite traduit ces exigences en une architecture découplée, documentée selon le modèle C4 : un module Odoo (frontend OWL), un proxy serveur garant des sessions et des ACLs, et un backend FastAPI où huit agents spécialisés collaborent au sein d'un graphe d'états LangGraph, appuyés par Redis et ChromaDB.

La réalisation a donné corps à cette architecture en allant au-delà du prototype : pipeline RAG complet sur un corpus de 4405 segments de documentation avec réordonnancement, seuil de pertinence et validation anti-hallucination ; couche LLM multi-fournisseur protégée par un circuit breaker ; streaming SSE de bout en bout diffusant en direct les étapes du pipeline et les jetons de la réponse ; formation adaptative fondée sur la répétition espacée SM-2 avec notation côté serveur ; tableaux de bord d'administration intégrés à Odoo ; et un durcissement de sécurité systématique — comparaison à temps constant, rotation de clés, limitation de débit par locataire, journal d'audit. Enfin, la validation a confirmé la robustesse de l'ensemble : 294 tests automatisés exécutés en intégration continue avec une couverture de code de 67 %, cinq flux fonctionnels validés de bout en bout, et des optimisations ramenant par exemple le guidage explicite de 9–14 secondes à 3,4–5,5 secondes et les requêtes documentaires répétées à un temps quasi nul.

Au-delà des résultats techniques, ce projet illustre une conviction : l'avenir des ERP ne réside pas seulement dans l'ajout de fonctionnalités, mais dans leur capacité à accompagner activement l'utilisateur. En transformant Odoo d'un outil que l'on subit en un environnement qui guide, démontre et forme, le système multi-agent développé ouvre la voie à une nouvelle génération d'assistance logicielle.

Plusieurs axes d'évolution prolongent naturellement ce travail :

- **Industrialisation en production** : déploiement des manifestes Kubernetes sur un cluster de production avec montée en charge automatique ;

- **Évaluation quantitative approfondie** : extension du jeu d'évaluation RAG (65 questions à ce jour) et intégration en régression continue, mesure du taux de rejet de la boucle de validation, campagne de tests de charge (k6 ou Locust) pour calibrer l'autoscaler, élévation de la couverture de code au-delà des 67 % actuels en ciblant les adaptateurs de fournisseurs LLM, et conduite d'une étude d'expérience utilisateur (SUS, questionnaires de satisfaction) auprès d'utilisateurs réels ;
- **Extension du corpus documentaire** : indexation continue des nouvelles versions de la documentation Odoo, des notes de version et de contenus internes à l'entreprise (procédures métier), avec ré-ingestion automatisée ;
- **Actions agentiques d'écriture** : permettre à l'assistant, sous confirmation explicite et dans le strict respect des ACLs, d'exécuter des actions de création ou de modification de données (préremplissage de formulaires, création de brouillons), transformant le guidage en véritable copilotage ;
- **Stratégie de fournisseur LLM et coûts d'exploitation** : le développement s'est appuyé sur NVIDIA NIM, dont le programme développeur offre un accès gratuit — un choix d'étudiant assumé, rendu sans conséquence par l'architecture indépendante du fournisseur (toute API compatible OpenAI convient). En production, le choix appartiendra à l'entreprise cliente selon son budget et ses contraintes de confidentialité — d'une API cloud commerciale à un déploiement local via Ollama, qui ramène le coût à celui du serveur d'inférence. Les optimisations décrites au chapitre IV — court-circuits heuristiques, caches, synthèse conditionnelle — réduisent directement cette facture puisqu'elles suppriment des appels ; il s'agit d'un paramètre de déploiement à dimensionner, non d'une limite du système ;
- **Validation Odoo Enterprise** : campagne de tests dédiée à l'édition Enterprise afin de certifier la compatibilité déjà assurée par conception ;
- **Expérience vocale avancée** : intégration d'une synthèse et d'une reconnaissance vocales cloud de haute qualité en complément de la Web Speech API, pour un assistant pleinement conversationnel ;
- **Ouverture aux canaux d'équipe** : exposition de l'assistant dans Slack ou Microsoft Teams afin d'assister les utilisateurs là où ils collaborent ;
- **Fonctionnalités optionnelles restantes du cahier des charges** : collaboration multi-utilisateur (partage de parcours guidés entre collègues), FAQ intelligente auto-générée à partir des questions fréquentes, et bac à sable de simulation pour des démonstrations sans risque sur des données réelles.

Bibliographie

Les références sont citées dans le texte sous la forme [n]. Les adresses électroniques et identifiants numériques (DOI) ont été consultés en juin 2026.

Travaux académiques à comité de lecture

- [1] Wooldridge, M., & Jennings, N. R. (1995). *Intelligent Agents : Theory and Practice*. The Knowledge Engineering Review, 10(2), 115–152. Cambridge University Press.
- [2] Kwon, O. B., & Lee, J. J. (2001). *A Multi-Agent Intelligent System for Efficient ERP Maintenance*. Expert Systems with Applications, 21(4). Elsevier. DOI : 10.1016/S0957-4174(01)00039-2.
- [3] Symeonidis, A. L., Kehagias, D. D., & Mitkas, P. A. (2003). *Intelligent Policy Recommendations on Enterprise Resource Planning by the Use of Agent Technology and Data Mining Techniques*. Expert Systems with Applications, 25(4), 589–602. Elsevier. DOI : 10.1016/S0957-4174(03)00099-X.
- [4] Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). *Large Language Model based Multi-Agents : A Survey of Progress and Challenges*. Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI-24), Survey Track, 8048–8057. DOI : 10.24963/ijcai.2024/890.
- [5] Vidgof, M., Bachhofner, S., & Mendling, J. (2023). *Large Language Models for Business Process Management : Opportunities and Challenges*. Business Process Management Forum (BPM 2023), Lecture Notes in Business Information Processing, vol. 490, 107–123. Springer. DOI : 10.1007/978-3-031-41623-1_7.
- [6] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct : Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR). arXiv :2210.03629.
- [7] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems 33 (NeurIPS). arXiv :2005.11401.
- [8] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP 2019. arXiv :1908.10084.
- [9] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30 (NeurIPS). arXiv :1706.03762.
- [10] Ebbinghaus, H. (1885). *Über das Gedächtnis. Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig. Traduction anglaise : Memory : A

Contribution to Experimental Psychology, Teachers College, Columbia University, 1913.

[11] Woźniak, P. A. (1990). *Optimization of Learning* (algorithme de répétition espacée SM-2). Mémoire de master, Université de technologie de Poznań — SuperMemo.

[12] Ye, J., Su, J., & Cao, Y. (2022). *A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling*. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), 4381–4390. DOI : 10.1145/3534678.3539081. Fondement de l'ordonnanceur FSRs.

Frameworks et outils d'orchestration

[13] LangChain. *Documentation officielle*. <https://python.langchain.com/docs/>

[14] LangGraph. *Documentation officielle*. <https://langchain-ai.github.io/langgraph/>

[15] CrewAI. *Documentation officielle*. <https://docs.crewai.com/>

[16] Microsoft AutoGen. *Documentation officielle*. <https://microsoft.github.io/autogen/>

[17] FastAPI. *Documentation officielle*. <https://fastapi.tiangolo.com/>

[18] ChromaDB. *Documentation officielle*. <https://docs.trychroma.com/>

Documentation Odoo

[19] Odoo S.A. *Odoo 19 Documentation*. <https://www.odoo.com/documentation/19.0/>

[20] Odoo S.A. *OWL — Odoo Web Library*. <https://github.com/odoo/owl>

Technologies, standards et ressources techniques

[21] Brown, S. *The C4 Model for Visualising Software Architecture*. <https://c4model.com/>

[22] Redis. *Documentation officielle*. <https://redis.io/docs/>

[23] WHATWG. *HTML Living Standard — Server-Sent Events*. <https://html.spec.whatwg.org/multipage/server-sent-events.html>

[24] TechEmpower. *Web Framework Benchmarks*. <https://www.techempower.com/benchmarks/>